

HTML文档主体

- **<body>**
 - 段落与文字
 - 列表
 - 表格
 - 多媒体与嵌入
 - 超链接
 - 表单
- **</body>**

表单

- 利用元素插入输入框

- 单选按钮radio

- 单选按钮radio只能从选项列表中选择一项，选项与选项之间是互斥的
 - 语法：<input type="radio" name="单选按钮所在的组名" value="单选按钮的取值"/>
 - **name和value属性是radio必要属性**，这两个属性必须要设置，用于向服务器提交数据：
 - `http://www.foo.com/?name=value`
 - **对于同一个问题的不同选项必须要设置一个相同的name属性值**，这样才能把这些选项归为同一个问题上

表单

- 利用元素插入输入框
— 单选按钮radio

在线代码测试

黑暗 黎明

显示效果

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5 </head>
6 <body>
7   <form name="form1" method="post" action="index.html">
8     性别:
9     <input type="radio" name="Question1" value="boy" checked />男
10    <input type="radio" name="Question1" value="girl" />女
11  </form>
12 </body>
13 </html>
```

性别： ☒ 男 ☐ 女

表单

- 利用<input>元素插入输入框

- 复选框checkbox

- 复选框checkbox可以从选项列表中选择一项或者多项
 - 语法：<input type="checkbox" name="复选框名称" value="复选框取值" checked="checked"/>
 - checked属性表示该选项在默认情况下已经被选中。不像单选按钮，对于复选框，一个选项列表中可以有多个复选框被选中
 - name和value属性是checkbox必要属性，这两个属性必须要设置，用于向服务器提交数据：
 - <http://www.foo.com/?name1=value1&name2=value2>
 - HTML中的复选框是没有文本的，需要加入label标签，并且用label标签的for属性指向复选框的id[语义化]

表单

- 利用元素插入输入框
 - 复选框checkbox

在线代码测试	黑暗	黎明	显示效果
<pre>1 <!DOCTYPE html> 2 <html xmlns="http://www.w3.org/1999/xhtml"> 3 <head> 4 <title></title> 5 </head> 6 <body> 7 <form name="form1" method="post" action="index.html"> 8 你喜欢的水果：
 9 <input id="checkbox1" type="checkbox" checked="checked" /> 10 <label for="checkbox1">苹果</label>
 11 <input id="checkbox2" type="checkbox" /> 12 <label for="checkbox2">香蕉</label>
 13 <input id="checkbox3" type="checkbox" /> 14 <label for="checkbox3">西瓜</label>
 15 <input id="checkbox4" type="checkbox" /> 16 <label for="checkbox4">凤梨</label> 17 </form> 18 </body> 19 </html></pre>			你喜欢的水果： ☒ 苹果 ☐ 香蕉 ☐ 西瓜 ☐ 凤梨

插入一个复选框，为其关联一个label，关联效果是点击文本也能改变复选框选中状态

表单

- 利用元素插入输入框

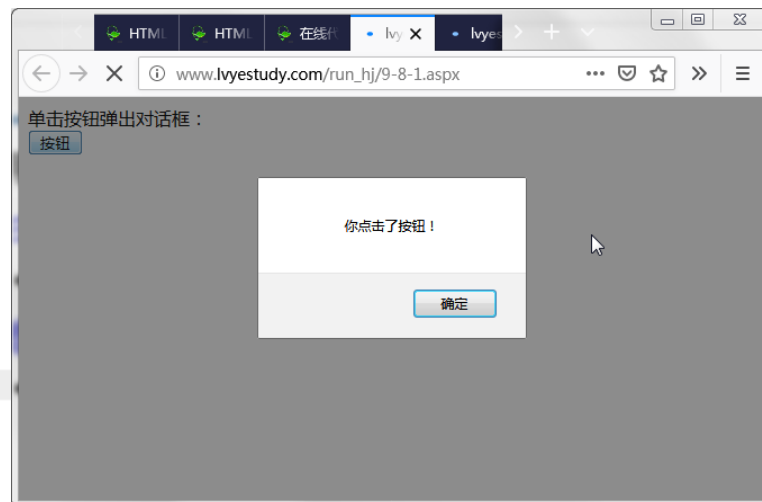
- 三种按钮:

- 普通按钮button、提交按钮submit、重置按钮reset

- 普通按钮button

- <input type="button" value="显示在按钮上的文字" onclick="JavaScript脚本程序"/>

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title></title>
5 </head>
6 <body>
7   单击按钮弹出对话框: <br/>
8   <input type="button" value="按钮" onclick="alert('你点击了按钮!')">
9 </body>
10</html>
```



表单

- 利用<input>元素插入输入框

- 提交按钮button

- <input type="submit" value="显示在按钮上的文字"/>
 - 将所在表单内的所有数据发送到服务器

- 重置按钮button

- <input type="reset" value="显示在按钮上的文字"/>
 - 将用户在所在表单输入的信息重置为默认值

表单

- 利用元素插入输入框

提交和重置 按钮示例

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title></title>
5 </head>
6 <body>
7   <form name="form1" method="get" action="http://foo.com">
8     账号:<input name="user" type="text" value="InText"/><br/>
9     密码:<input name="password" type="password" value="123"/><br/>
10    <input type="submit" value="提交"/>
11    <input type="reset" value="重置"/>
12  </form>
13  <input name="text2" type="text" value="OutText"/>
14 </body>
15 </html>
```

账号:

密码:

账号:

密码:

账号:

密码:

备注: 提交和重置按钮只针对当前所在form标签内的表单元素内容有效，对当前所在form标签之外的表单元素无效

表单

- 利用元素插入输入框

- 图片域image

-

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title></title>
5 </head>
6 <body>
7   <form name="form1" method="get" action="http://foo.com">
8     账号:<input name="user" type="text" value="InText"/><br/>
9     密码:<input name="password" type="password" value="123"/><br/>
10    <input type="submit" value="提交"/>
11    <input type="reset" value="重置"/>
12    <input type="image" src="http://www.lvvestudy.com/App_images/favicon.ico"/>
13  </form>
14  <input name="text2" type="text" value="OutText"/>
15 </body>
16 </html>
```

账号: InText

密码: ●●●

提交

重置

OutText

点击会发生
什么呢?

答案

www.foo.com/?user=InText&password=123&x=19&y=21

和普通提交按钮一样, 提交信息增加了鼠标点击图片时的相对焦点坐标

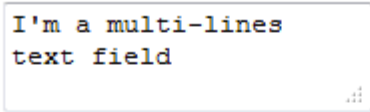
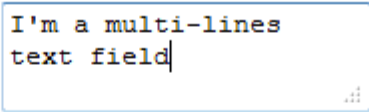
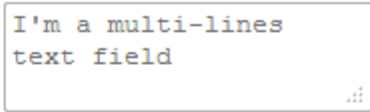
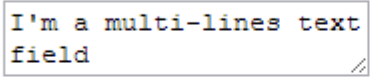
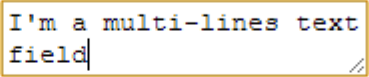
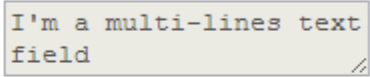
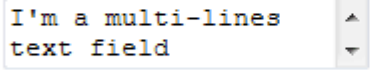
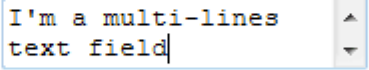
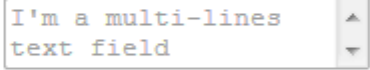
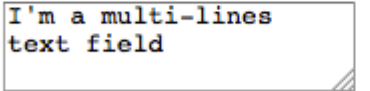
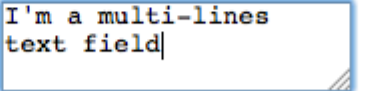
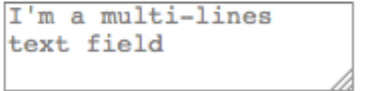
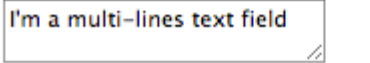
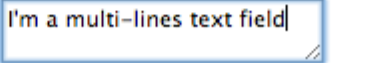
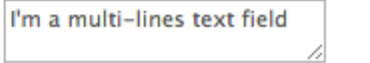
School of Cyber Security, BUPT #9

表单

- 利用<textarea>元素插入多行文本输入控件

语法: <textarea rows="行数" cols="列数">多行文本框内容</textarea>

```
1 | <textarea cols="30" rows="10"></textarea>
```

	Default	Focus	Disabled
Firefox (Win7)			
Chrome (Win7)			
IE (Win7)			
Firefox (Mac OSX)			
Chrome (Mac OSX)			

表单

- 利用<select>和<option>元素插入下拉选择框
 - 下拉列表由<select>和<option>这2个标签配合使用
 - 利用<optgroup>设置选项分组

```
<select id="groups" name="groups">
  <optgroup label="fruits">
    <option>Banana</option>
    <option selected>Cherry</option>
    <option>Lemon</option>
  </optgroup>
  <optgroup label="vegetables">
    <option>Carrot</option>
    <option>Eggplant</option>
    <option>Potato</option>
  </optgroup>
</select>
```



Cherry	▼
fruits	
Banana	
Cherry	
Lemon	
vegetables	
Carrot	
Eggplant	
Potato	

可以设置multiple属性，通过“Ctrl/Shift+鼠标左键”实现多选
向服务器提交数据格式：<http://www.foo.com/?name=value1&name=value2>

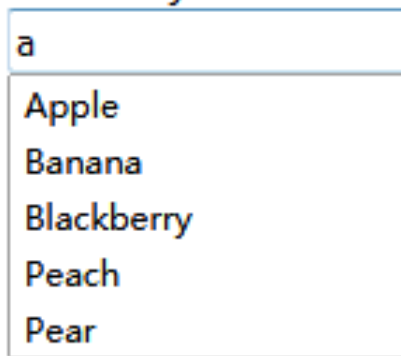
表单

- 利用<datalist>元素插入自动补全输入框

- 自动补全输入框提供一个下拉框向用户展示的，在输入框中输入可能匹配的内容
- 使用<datalist>元素来为表单小部件提供建议的、自动完成的值，并使用<option>指定显示的值，使用list属性将数据列表绑定到一个文本域(通常是一个<input>元素)，关联后选项内容用于自动完成用户输入的文本

```
<label for="myFruit">What's your favorite fruit?</label>
<input type="text" name="myFruit" id="myFruit" list="mySuggestion">
<datalist id="mySuggestion">
  <option>Apple</option>
  <option>Banana</option>
  <option>Blackberry</option>
  <option>Blueberry</option>
  <option>Lemon</option>
  <option>Lychee</option>
  <option>Peach</option>
  <option>Pear</option>
</datalist>
```

What's your favorite fruit?



a

- Apple
- Banana
- Blackberry
- Peach
- Pear

表单

- 利用<fieldset>和<legend>元素创建控件组
 - fieldset对表单控件进行分组，一个表单可以有多个控件组
 - legend用于对每组内容进行描述
 - 用于定义语义结构良好的表单

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title></title>
5 </head>
6 <body>
7   <form action="test.php" method="post">
8     <fieldset>
9       <legend>Which one is most important in webdev?</legend>
10      <input type="radio" id="radio1"> <label for="radio1">HTML</label><br/>
11      <input type="radio" id="radio2"> <label for="radio2">CSS</label><br/>
12      <input type="radio" id="radio3"> <label for="radio3">Javascript</label>
13    </fieldset>
14  </form>
15 </body>
16 </html>
```

显示效果

Which one is most important in webdev? —

☐ HTML

☐ CSS

☐ Javascript

legend

fieldset

表单

- 利用<label>元素为元素设置标签
— 用于定义语义结构良好的表单

```
<form action="demo.php" method="post">
<fieldset>
<legend>请输入您的个人信息</legend>
<label for="name">姓名: </label>
<input type="text" id="name" name="user_name"><br />
性别:
<input type="radio" id="gender_male" name="gender" value="Male">
<label for="gender_male">男性</label>
<input type="radio" id="gender_female" name="gender" value="Female">
<label for="gender_female">女性</label>
</fieldset>
</form>
```

请输入您的个人信息

姓名: 特朗普

性别: ☒ 男性 ☐ 女性

HTML文档主体

- **<body>**

- 段落与文字
- 列表
- 表格
- 多媒体与嵌入
- 超链接
- 表单

- **</body>**



相关知识：文档与网站架构

除了定义页面的各个部分（例如“段落”或“图像”）外，HTML也用于定义网站的区域（例如“标题”，“导航菜单”，“主要内容列”）。此部分探讨如何规划基本网站结构，以及如何编写HTML以表示此结构。

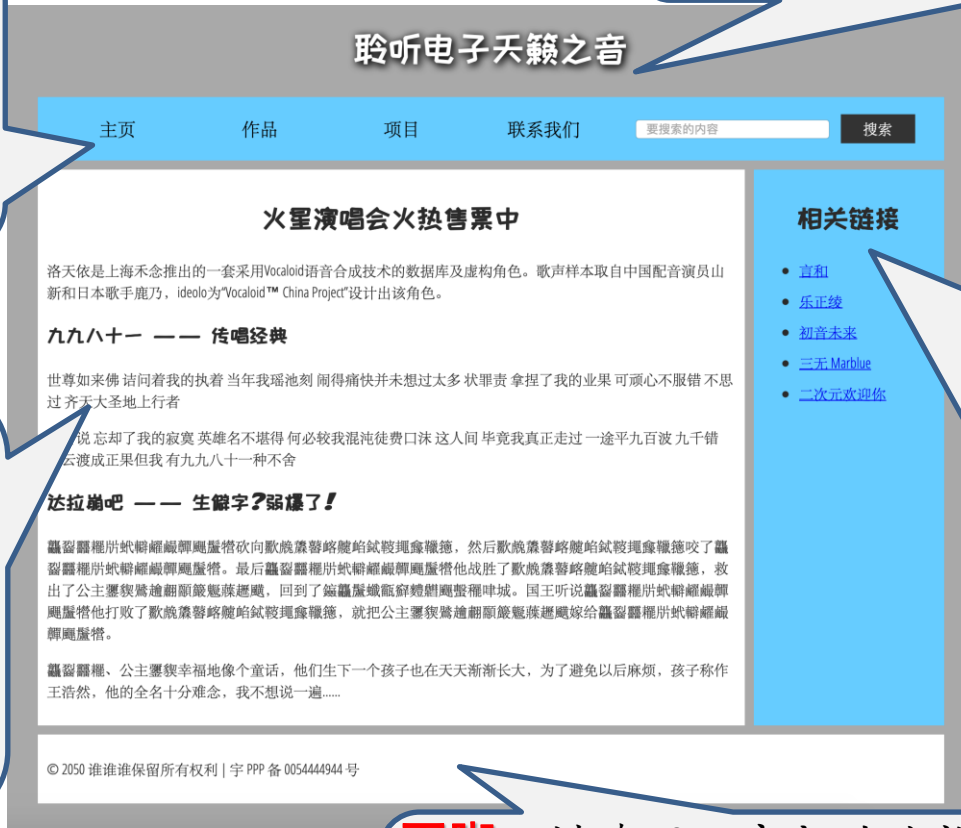
文档与网站架构

• 网页文档的基本组成部分

导航栏：指向网站各个主要区段的超链接。通常用菜单按钮、链接或标签页表示

页眉：通常横跨于整个页面顶部有一个大标题和/或一个标志，这是网站的主要一般信息，通常存在于所有网页

主要内容：中心的大部分区域是网页的独有内容，如视频、文章、地图、新闻等，这些内容是网站的一部分，会因页面而异



侧边栏：一些外围信息、链接、引用、广告等。通常与主要内容相关（例如一个新闻页面上，侧边栏可能包含作者信息或相关文章链接），还可能存在其他的重复元素，如辅助导航系统

页脚：横跨页面底部的狭长区域，和标题一样，页脚是放置公共信息（如版权声明或联系方式）的，一般使用较小字体，且通常为次要内容

文档与网站架构

- 网页文档的基本组成部分

- 以上简单示例不是很精美，但是足够说明网页典型布局方式。尽管通过CSS可以使任意页面元素放置在页面的不同区域来做成想要的样子，但我们要**敬畏语义，需要正确选用元素实现网页布局**

- 这是因为视觉效果并不是一切。虽然可以修改最重要内容（例如导航菜单和相关链接）的颜色、字体大小来吸引用户的注意，但是这对视障人士是无效的，“粉红色”和“大字体”对他们并不奏效

文档与网站架构

- 网页文档的基本组成部分

- HTML 代码中可根据功能来为区段添加标记，可使用元素来无歧义地表示上文所讲的内容区段，**屏幕阅读器**等辅助技术可以识别这些元素，并帮助执行“找到主导航”或“找到主内容”等任务
- 为了实现语义化标记，HTML 提供了明确这些区域的专用标签，如：
 - 页眉<header>、导航栏<nav>
 - 主内容<main>，其中可包括各种内容区域，如<article>、<section>、<div>等
 - 侧边栏<aside>，经常嵌套在<main>中、页脚<footer>

文档与网站架构

• 网页文档区域布局相关元素

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="utf-8">
5     <title>二次元俱乐部</title>
6   </head>
7   <body>
8     <header><!-- 本站所有网页的统一主标题 -->
9       <h1>聆听电子天籁之音</h1>
10    </header>
11    <nav><!-- 本站统一的导航栏 -->
12      <ul>
13        <li><a href="#">主页</a></li>
14        <!-- 共n个导航栏项目，省略..... -->
15      </ul>
16      <form><!-- 搜索栏是站点内导航的一个非线性的方式。 -->
17        <input type="search" name="q" placeholder="要搜索的内容">
18        <input type="submit" value="搜索">
19      </form>
20    </nav>
21    <main><!-- 网页主体内容 -->
22      <article>
23        <!-- 此处包含一个 article（一篇文章），内容略..... -->
24      </article>
25      <aside><!-- 侧边栏在主内容右侧 -->
26        <h2>相关链接</h2>
27        <ul>
28          <li><a href="#">这是一个超链接</a></li>
29          <!-- 侧边栏有n个超链接，略略略..... -->
30        </ul>
31      </aside>
32    </main>
33    <footer><!-- 本站所有网页的统一页脚 -->
34      <p>© 2050 某某保留所有权利</p>
35    </footer>
36  </body>
37 </html>
```

聆听电子天籁之音

- [主页](#)

相关链接

- [这是一个超链接](#)

© 2050 某某保留所有权利

页面不像我们想的那样？那是CSS布局阶段需要解决的事情，HTML侧重在解决网页结构

文档与网站架构

- 网页文档区域布局相关元素

- `<main>` 存放每个页面独有的内容，每个页面上**只能用一次 `<main>`**，**且直接位于 `<body>` 中**，最好不要把它嵌套进其它元素
- `<article>` 包围的内容即一篇文章，与页面其它部分无关（比如一篇博文）
- `<section>` 与 `<article>` 类似，但 `<section>` 更适用于组织页面使其**按功能分块**（比如迷你地图、一组文章标题和摘要）

- 网页组织一般的最佳用法如下：

- 以标题作为开头，也可以把一篇 `<article>` 分成若干部分并分别置于不同的 `<section>` 中，也可以把一个区段 `<section>` 分成若干部分并分别置于不同的 `<article>` 中，取决于上下文

文档与网站架构

- 网页文档区域布局相关元素
 - `<aside>` 包含一些间接信息
 - 术语条目、作者简介、相关链接等等
 - `<header>` 是简介形式的内容
 - 如果它是 `<body>` 的子元素，那么就是网站的全局页眉，如果它是 `<article>` 或 `<section>` 的子元素，那么它是这些部分特有的页眉（此 `<header>` 非彼标题）
 - `<nav>` 包含页面主导航功能，其中不应包含二级链接等内容
 - `<footer>` 包含页面的页脚部分

文档与网站架构

- 网页文档区域布局相关元素

- 有时会发现，对于一些要组织的项目或要包装的内容，现有的语义元素均不能很好对应；有时候可能只想将一组元素作为一个单独的实体来用单一的CSS 或 JavaScript来修饰来响应。为了应对这种情况，HTML提供了 `<div>` 和 `<span>`这类无语义元素：

- `<span>` 是一个内联的（inline）无语义元素，最好只用于无法找到更好的语义元素来包含内容时，或者不想增加特定的含义时
 - `<div>` 是一个块级无语义元素，应仅用于找不到更好的块级元素时，或者不想增加特定的意义时

文档与网站架构

- 规划一个简单的网站

- 在完成页面内容的规划后，一般应按部就班地规划整个网站的内容，要可能带给用户最好的体验，需要哪些页面、如何排列组合这些页面、如何互相链接等问题不可忽略，这些工作称为信息架构设计
- 在大型网站中，大多数规划工作都可以归结于此，而对于一个只有几个页面的简单网站，规划设计过程可以更简单，更有趣！

文档与网站架构

- 规划一个简单的网站

- Step1: 时刻记住，大多数（不是全部）页面会使用一些**相同的元素**，例如导航菜单以及页脚内容，若网站为商业站点，不妨在所有页面的页脚都加上**联系方式**，请记录这些对所有页面都通用的内容：

通用内容

页眉：标题和 logo

页脚：联系方式和版权声明

① 术语、状态

② 站点语言选择

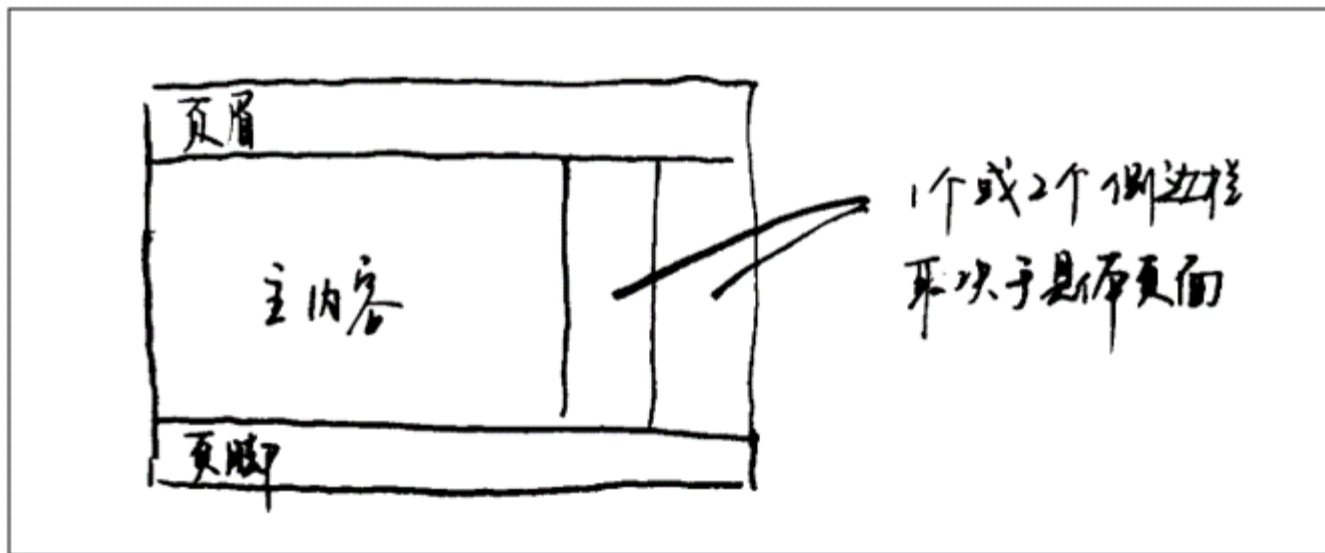
③ 无障碍访问策略

超链接

文档与网站架构

- 规划一个简单的网站

- Step2: 为页面结构绘制草图，记录每一块的作用：



文档与网

- 规划一个简单的网站

- Step3: 对于期望添加进站点的所有其它（通用内容以外的）内容展开头脑风暴，直接罗列出来：

搜索: 航班.

酒店 / 其它住宿方式

交通

待办事项

限时优惠信息

流行的假期出游套餐

例如: 迪士尼世界.

欢乐谷
主题公园

搜索结果

出行国家信息

住宿 / 出入境资料

信用卡 / 货币.

现金 / 汇率

语言

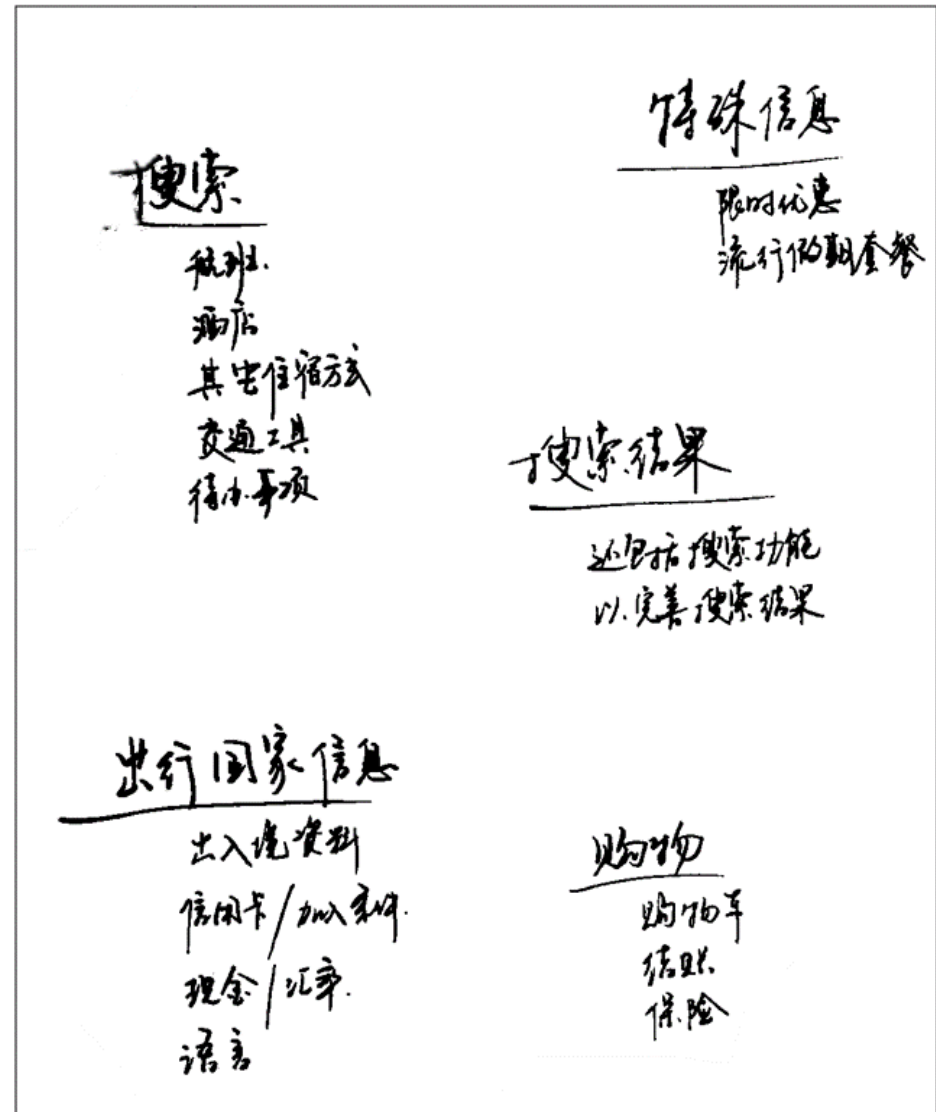
保险

购买纪念品

其他物品

文档与网站架构

- 规划一个简单的网站
 - Step4: 试着对这些内容进行分组，这样可以让你了解哪些内容可以放在同一个页面:

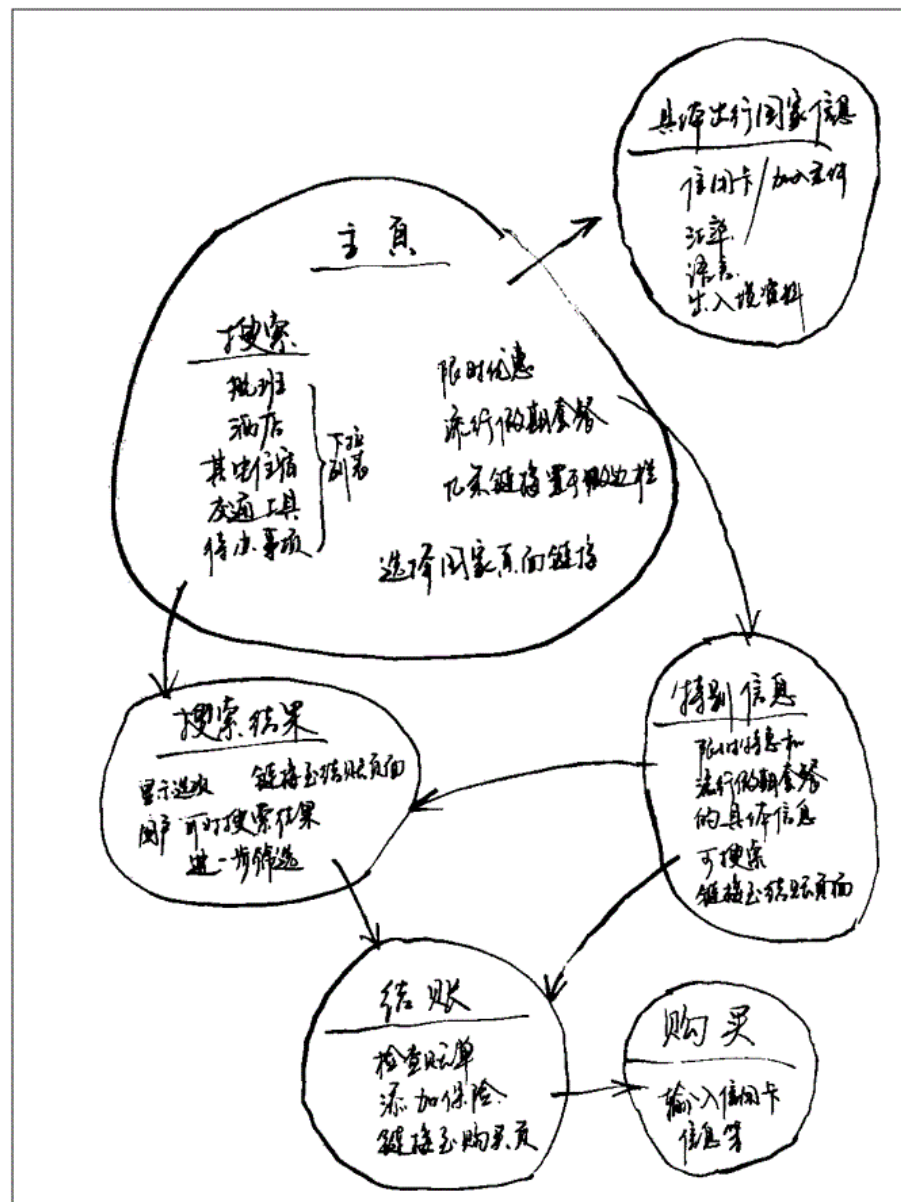


文档与网站架构

- 规划一个简单的网站

- Step5: 绘制一个站点地图的草图，使用一个气泡代表网站的一个页面，并绘制连线来表示页面间的一般工作流

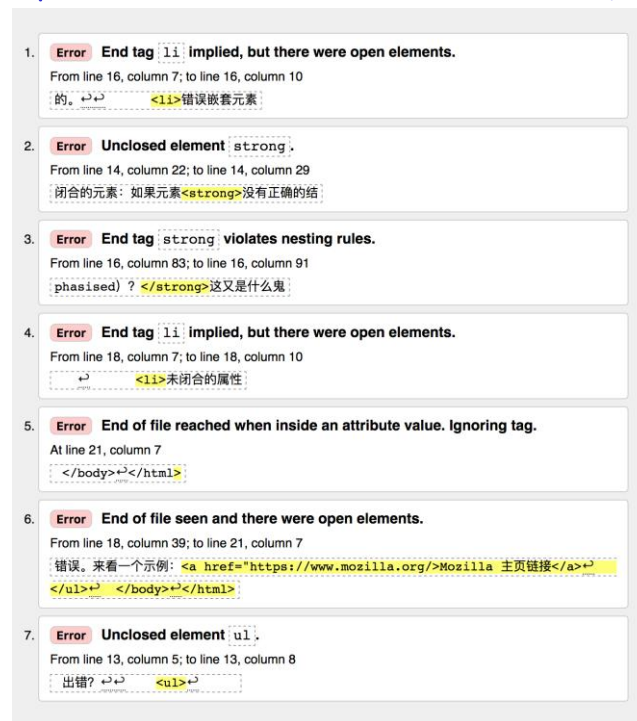
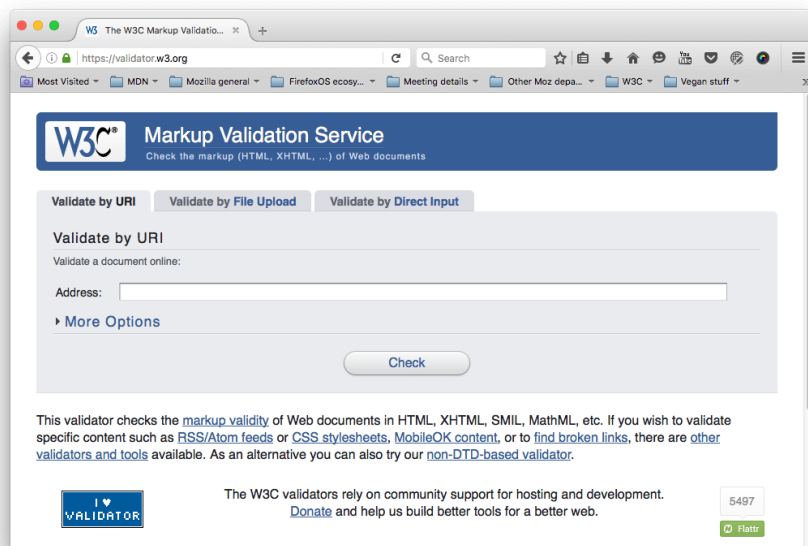
- 主页面一般置于中心，且链接到其他大多数页面，小型网站的大多数页面都可以从主页的导航栏中链接跳转，也可记录下内容的显示方式



HTML验证

- HTML 验证的问题

- HTML语法非常宽松，如何从非常庞大、复杂的HTML文档发现语法错误？
- 最好方法就是让HTML页面通过 **Markup Validation Service**（一个由 W3C 创立并维护的标记验证服务）



把一个HTML文档加载至该服务并运行后，会返回一个错误报告，给出语法错误清单

课程小结

<!DOCTYPE html>

<html>

<head>

添加网页标题: <title>元素

添加网页元信息: <meta>元素

应用CSS和JavaScript: <link>元素和<script>元素

</head>

<body>

段落与文字: <p><h1>-<h6><div>, ...

列表: <dl>, <di><dd>

表格: <table><tr><td>, <th><caption>, <thead><tfoot><tbody>

多媒体嵌入: <video><audio>, <embed><object>, <iframe>

超链接: <a>

表单: <form>, <input><textarea><select><datalist>, <fieldset><legend><label>

</body>

</html>

重点

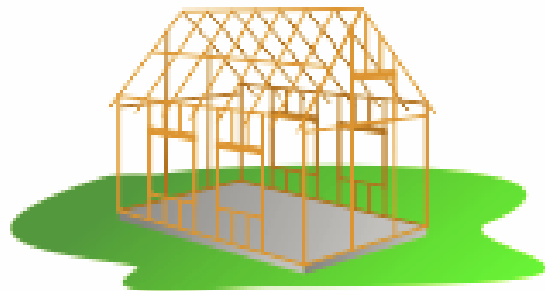
- 学习HTML的重点不在于掌握了多少标签，而在于掌握标签的语义，知道如何编写一个语义结构良好的页面
 - 便于开发调试和后期维护
 - 便于搜索引擎优化
 - 便于后期设置CSS样式

层叠样式表CSS (Cascading Style Sheets, CSS)



什么是CSS

- CSS是**C**ascading **S**tyle **S**heet（层叠样式表）的简称
- CSS是一种**样式表语言**（HTML是标记语言），用来描述HTML或XML（包括如SVG、XHTML之类的XML分支语言）文档的呈现
- CSS描述了在屏幕、纸质、音频等其它媒体上的元素应该如何被渲染的问题



The house frame
"HTML"



The bricks, tiles and paint
"CSS"

CSS的发展历史

- 在Tim Berners-Lee发明万维网的那个年代，HTML语言是唯一用于给文本添加结构的语言。作者可以通过声明“这是一个标题”（利用h1标签）或“这是一个段落”（利用p标签）的方式来标记文本
- 随着Web逐渐流行起来，网页设计者们开始寻求为网页增添布局的可能性，为此，当时的浏览器厂商们（网景公司和微软公司）发明了一些新的HTML标签（比如等），以引入了新的布局——而非新的结构。这也导致了原本用于进行文本的结构化的标签（比如<table>等）越来越多地被误用于进行页面的布局
- 各个浏览器厂商为了在竞争中取得更多的市场份额，在浏览器实现中都加入了很多各自相互不兼容的样式标记，这使得同一个HTML文档很难在多个浏览器中兼容，“您需要使用某某浏览器来浏览本页面”成为当时常见于许多网站的声明
- CSS的发明正是为了改善这一状况，它为Web设计师们提供了完善的、所有浏览器都支持的布局能力。而且，文档的表现样式与内容的分离，也令网站维护容易了许多

CSS的发展历史

1994 年 **Håkon Wium Lie** 为 HTML 样式提出了 CSS 的最初建议。Bert Bos 当时正在设计一个叫做 Argo 的浏览器，他们决定一起合作设计 CSS，于是形成了 CSS 的最初版本。1994 年 **Håkon Wium Lie** 在芝加哥的一次会议上第一次正式提出了 CSS 的建议，1995 年他与 **Bert Bos** 一起再次展示这个建议。当时 W3C 刚刚建立，W3C 对 CSS 的发展很感兴趣，它为此专门组织了一次讨论会。同期提出的类似提案有 10 多种。



Håkon Wium Lie, 12 December 1995

16 June 2017	Håkon & Bert present 'CSS Reel' at CSS Day
January 2017	IDPF & W3C join to develop e-book standards
17 December 2016	20th anniversary of CSS, first CSS info e-book, a MOOC on W3C
7 November 2016	CSS info e-book, a MOOC on W3C
12 April 2016	First draft of 2nd revision of CSS level 2
12 November 2014	Philae lands on comet 67P
23 June 2013	W3C creates the Digital Publishing IG
13 December 2012	W3C Recommendation for WOFF 1.0
3 April 2012	Final version of JREQ
18 June 2011	MapCSS, CSS-like styles for maps
11 March 2011	Fukushima Daiichi nuclear disaster
27 July 2010	First draft of WOFF 1.0
April 2010	caniuse.com (Can I use?) browser support tables
August 2009	LESS CSS pre-processor
2 June 2009	jQuery CSS, CSS-like styles for jQuery
26 May 2008	Microsoft & Monotype open the EOT format
October 2007	EPUB 2.0 standard released
18 December 2006	Fuji CSS Validator released (more)
October 2005	Sass CSS pre-processor
17/18 October 2006	W3C Print Symposium in Heidelberg
4 October 2006	Qt style sheets, CSS-like styles for Qt
13 April 2005	Acid2 test by Håkon Lie & Ian Hickson
13-29 August 2004	Summer Olympics in Athens
11 October 2001	W3C Recommendation of XSL 1.0
4 September 2001	W3C Recommendation of SVG 1.0
15 January 2001	Start of Wikipedia
7 May 2003	Dave Shea launches the CSS Zen Garden
2 August 2002	First draft of revised CSS level 2
22 June 1999	First 3 CSS3 drafts: Color Profiles, Multi-column layout, and Paged Media
29 January 1999	CSS1 Test Suite (press release) with Todd Palmer's Box Acid Test
11 January 1999	W3C revises CSS level 1 Recommendation
4 September 1998	Google is founded
12 May 1998	W3C publishes CSS level 2 Recommendation; read the press release
30 March 1998	First beta of CSS Validator announced
4 November 1997	First Public Working Draft of CSS level 2 (CSS2); read the press release
17 December 1996	W3C Recommendation of CSS level 1; read the press release
13 August 1996	MSIE 3 browser, with support for CSS
3 July 1996	Birth of Dolly the sheep
27 November 1995	Experimental support for CSS in W3C's Arena browser
6-7 November 1995	W3C Workshop on Style Sheets in Rocquencourt, France
24 August 1995	Release of Windows 95
10 October 1994	Håkon's e-mail to www-w3@w3.org

详细信息参见：20 Years of CSS, <https://www.w3.org/Style/CSS20/>

CSS的发展历史

1996年12月W3C终于推出了CSS规范的第一版本，CSS1.0。这一规范立即引起了各方的积极响应，随即Microsoft公司和Netscape公司纷纷表示自己的浏览器能够支持CSS1.0，从此CSS技术的发展几乎一马平川



Netscape Navigator 3.0，当年的浏览器王者

10 October 1994	Hickson's e-mail to www-talk@w3.org
27 November 1995	Experimental support for CSS in W3C's Arena browser
6-7 November 1995	W3C Workshop on Style Sheets in Rocquencourt, France
24 August 1995	Release of Windows 95
13 August 1996	MSIE 3 browser, with support for CSS
3 July 1996	Birth of Dolly the sheep
17 December 1996	W3C Recommendation of CSS level 1; read the press release
22 June 1999	First 3 CSS3 drafts: Color Profiles, Multi-column layout, and Paged Media
29 January 1999	CSS1 Test Suite (press release) with Todd Palmer's Box Acid Test
11 January 1999	W3C revises CSS level 1 Recommendation
4 September 1998	Google is founded
12 May 1998	W3C publishes CSS level 2 Recommendation; read the press release
30 March 1998	First beta of CSS Validator announced.
4 November 1997	First Public Working Draft of CSS level 2 (CSS2); read the press release
11 October 2001	W3C Recommendation of XSL 1.0
4 September 2001	W3C Recommendation of SVG 1.0
15 January 2001	Start of Wikipedia
7 May 2003	Dave Shea launches the CSS Zen Garden
2 August 2002	First draft of revised CSS level 2.
13 April 2005	Acid2 test by Hickson Lie & Ian Hickson
13-29 August 2004	Summer Olympics in Athens
18 October 2006	Fuji CSS Validator released (more)
17/18 October 2006	Sass CSS pre-processor
4 October 2006	W3C Print Symposium in Heidelberg Qt style sheets, CSS-like styles for QtUIs
26 May 2008	Microsoft & Monotype open the EOT format
October 2007	EPUB 2.0 standard released
2 June 2009	LESS CSS pre-processor jQuery CSS, CSS-like styles for jQuery
27 July 2010	First draft of WOFF 1.0
April 2010	Can I use? browser support tables
18 June 2011	MapCSS, CSS-like styles for maps
11 March 2011	Fukushima Daiichi nuclear disaster
3 April 2012	Final version of jQuery
23 June 2013	W3C creates the Digital Publishing IG
13 December 2012	W3C Recommendation for WOFF 1.0
12 April 2016	First draft of 2nd revision of CSS level 2
17 December 2016	10th anniversary of CSS level 1
7 November 2016	CSS intro course, a MOOC on W3C
16 June 2017	Hickson & Bert present CSS Reel at CSS Day
January 2017	IDPF & W3C join to develop e-book standards

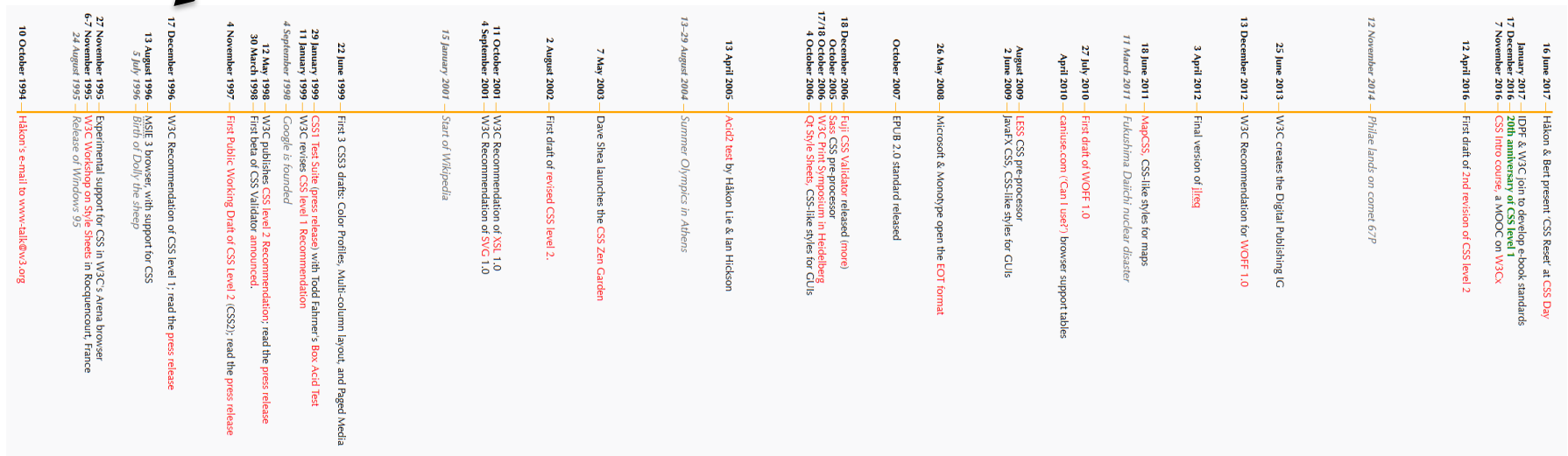
详细信息参见：20 Years of CSS, <https://www.w3.org/Style/CSS20/>

CSS的发展历史

1996年12月W3C终于推出了CSS规范的第一版本，**CSS1.0**。这一规范立即引起了各方的积极响应，随即Microsoft公司和Netscape公司纷纷表示自己的浏览器能够支持CSS1.0，从此CSS技术的发展几乎一马平川

CSS 1.0

- **选择器**：用来定义希望应用样式的HTML元素或者标签
- **样式属性**：主要包括字体、背景、尺寸、布局等，用来定义你希望改变的样式
- **伪类**：用来定义样式如何**动态变化**，如鼠标悬停等



A detailed timeline of CSS development from 1994 to 2017. The timeline is presented as a horizontal bar with a vertical axis on the left showing dates. The bar is divided into segments by vertical lines, each representing a significant event in the history of CSS. The events include the release of CSS1, the creation of the CSS2.1 specification, the release of CSS3, and the release of CSS4. The timeline ends with the release of CSS4 in 2017.

10 October 1994	Hickson's e-mail to www-ta1k@w3.org
27 November 1995	Experimental support for CSS in W3C's Arena browser
6-7 November 1995	W3C Workshop on Style Sheets in Rocquencourt, France
24 August 1995	Release of Windows 95
13 August 1996	MSIE 3 browser, with support for CSS
3 July 1996	Birth of Dolly the sheep
17 December 1996	W3C Recommendation of CSS level 1; read the press release
22 June 1999	First 3 CSS3 drafts: Color Profiles, Multi-column layout, and Paged Media
29 January 1999	CSS1 Test Suite (press release) with Todd Palmer's Box Acid Test
11 January 1999	W3C revises CSS level 1 Recommendation
4 September 1998	Google is founded
12 May 1998	W3C publishes CSS level 2 Recommendation; read the press release
30 March 1998	First beta of CSS Validator announced.
4 November 1997	First Public Working Draft of CSS level 2 (CSS2); read the press release
11 October 2001	W3C Recommendation of XSL 1.0
4 September 2001	W3C Recommendation of SVG 1.0
15 January 2007	Start of Wikipedia
7 May 2003	Dave Shea launches the CSS Zen Garden
2 August 2002	First draft of revised CSS level 2.
13-29 August 2004	Summer Olympics in Athens
13 April 2005	Acid2 test by Hickson Lie & Ian Hickson
18 December 2006	Fuji CSS Validator released (more)
17/18 October 2006	Sass CSS pre-processor
4 October 2006	W3C Print Symposium in Heidelberg Qt style sheets, CSS-like styles for GLUs
26 May 2008	Microsoft & Monotype open the EOT format
October 2007	EPUB 2.0 standard released
August 2009	LESS CSS pre-processor
2 June 2009	jQuery CSS, CSS-like styles for jQuery
27 July 2010	First draft of WOFF 1.0
April 2010	caniuse.com (Can I use?) browser support tables
18 June 2011	MapCSS, CSS-like styles for maps
11 March 2011	Fukushima Daiichi nuclear disaster
3 April 2012	Final version of jQuery
13 December 2012	W3C Recommendation for WOFF 1.0
23 June 2013	W3C creates the Digital Publishing IG
16 June 2017	Hickson & Bert present 'CSS Reel' at CSS Day
January 2017	IDPF & W3C join to develop e-book standards
17 December 2016	10th anniversary of CSS level 1
7 November 2016	CSS Info course, a MOOC on W3C
12 April 2016	First draft of 2nd revision of CSS level 2

详细信息参见：20 Years of CSS, <https://www.w3.org/Style/CSS20/>

CSS的发展历史

- 1998 年 W3C 发布了 CSS 的第二个版本，CSS2.0。CSS2 规范是基于 CSS1 设计的，其包含了 CSS1 所有的功能，并扩充和改进了很多更加强大的属性。这些新功能极大的增强了 CSS 和 HTML 的表现能力，使得 Web 应用拥有和本地应用程序一样的表现力，甚至是更好的用户体验。
- 虽然 CSS 规范有如此之多的优点，但其推广和采用却一直很艰难。浏览器支持的现状和各种问题使得 W3C 不得不在 2007 年最终修订 CSS2 为 CSS2.1，删除和修改一些属性和行为，使其更接近当前浏览器支持和实现。目前市面上主流的浏览器都已经支持了 CSS2.1 中 99.9% 的功能，CSS2.1 是推荐标准，也是本课程的主要学习内容



16 June 2017	Håkon & Bert present 'CSS Reel' at CSS Day
January 2017	IDPF & W3C join to develop e-book standards
17 December 2016	W3C celebrates 20th anniversary of CSS
7 November 2016	CSS Info course, a MOOC on W3C's
12 April 2016	First draft of 2nd revision of CSS level 2
12 November 2014	Philae lands on comet 67P
23 June 2013	W3C creates the Digital Publishing IG
13 December 2012	W3C Recommendation for WOFF 1.0
3 April 2012	Final version of JReq
18 June 2011	MapCSS: CSS-like styles for maps
11 March 2011	Fukushima Daiichi nuclear disaster
27 July 2010	First draft of WOFF 1.0
April 2010	caniuse.com (Can I use?) browser support tables
August 2009	LESS: CSS pre-processor
2 June 2009	jQuery CSS, CSS-like styles for jQuery
26 May 2008	Microsoft & Monotype open the EOT format
October 2007	EPUB 2.0 standard released
18 December 2006	Fugu CSS Validator released (more)
October 2005	Sass CSS pre-processor
17/18 October 2006	W3C Print Symposium in Heidelberg
4 October 2006	Qt style sheets, CSS-like styles for Qt
13 April 2005	Acid? test by Håkon Lie & Ian Hickson
13-20 August 2004	Summer Olympics in Athens
11 October 2001	W3C Recommendation of XSL 1.0
4 September 2001	W3C Recommendation of SVG 1.0
15 January 2001	Start of Wikipedia
7 May 2003	Dave Shea launches the CSS Zen Garden
2 August 2002	First draft of revised CSS level 2
11 October 2001	W3C Recommendation of XSL 1.0
4 September 2001	W3C Recommendation of SVG 1.0
22 June 1999	First 3 CSS3 drafts: Color Profiles, Multi-column layout, and Paged Media
29 January 1999	CSS1 Test Suite (press release) with Todd Palmer's Box Acid Test
11 January 1999	W3C revises CSS level 1: Recommendation
4 September 1998	Google is founded
12 May 1998	W3C publishes CSS level 2 Recommendation; read the press release
30 March 1998	First beta of CSS Validator announced.
4 November 1997	First Public Working Draft of CSS level 2 (CSS2); read the press release
17 December 1996	W3C Recommendation of CSS level 1; read the press release
13 August 1996	MSE 3 browser, with support for CSS
3 July 1996	Birth of Dolly the sheep
27 November 1995	Experimental support for CSS in W3C's Arena browser
6-7 November 1995	W3C Workshop on Style Sheets in Rocquencourt, France
24 August 1995	Release of Windows 95
10 October 1994	Håkon's e-mail to www-talk@w3.org

详细信息参见：20 Years of CSS, <https://www.w3.org/Style/CSS20/>

CSS的发展历史

- 2001 年 5 月，W3C 就着手开始准备开发 CSS 第三版规范。
- CSS 没有像软件版本那样的“版本号”，而是一个持续发展的标准。因此，CSS 版本并不直接体现在代码中，也没有“查看版本”的方式。现代的网页开发通常是混合使用 CSS2.1、CSS3 以及新功能模块的组合，具体取决于你所使用的 CSS 特性以及浏览器的支持情况。
- 比如，CSS3 引入了一些新特性，如 border-radius（圆角）、box-shadow（阴影）和 flexbox（弹性盒模型）。如果你使用了这些特性，说明你正在使用 CSS3 的模块。

16 June 2017	Håkon & Bert present 'CSS Reel' at CSS Day
January 2017	IDPF & W3C join to develop e-book standards
17 December 2016	10th anniversary of CSS level 1
7 November 2016	CSS Info course, a MOOC on W3C's
12 April 2016	First draft of 2nd revision of CSS level 2
12 November 2014	Philae lands on comet 67P
23 June 2013	W3C creates the Digital Publishing IG
13 December 2012	W3C Recommendation for WOFF 1.0
3 April 2012	Final version of JReq
18 June 2011	MapCSS: CSS-like styles for maps
11 March 2011	Fukushima Daiichi nuclear disaster
27 July 2010	First draft of WOFF 1.0
April 2010	caniuse.com (Can I use?) browser support tables
August 2009	LESS: CSS pre-processor
2 June 2009	jQuery CSS, CSS-like styles for jQuery
26 May 2008	Microsoft & Monotype open the EOT format
October 2007	EPUB 2.0 standard released
18 December 2006	Fuji CSS Validator released (more)
17/18 October 2006	Sass CSS pre-processor
4 October 2006	W3C Print Symposium in Heidelberg Qt style sheets, CSS-like styles for Qt
13-29 August 2004	Summer Olympics in Athens
13 April 2005	Acid2 test by Håkon Lie & Ian Hickson
7 May 2003	Dave Shea launches the CSS Zen Garden
2 August 2002	First draft of revised CSS level 2
11 October 2001	W3C Recommendation of XSL 1.0
4 September 2001	W3C Recommendation of SVG 1.0
15 January 2001	Start of Wikipedia
22 June 1999	First 3 CSS3 drafts: Color Profiles, Multi-column layout, and Paged Media
29 January 1999	CSS1 Test Suite (press release) with Todd Palmer's Box Acid Test
11 January 1999	W3C revises CSS level 1 Recommendation
4 September 1998	Google is founded
12 May 1998	W3C publishes CSS level 2 Recommendation; read the press release
30 March 1998	First beta of CSS Validator announced.
4 November 1997	First Public Working Draft of CSS level 2 (CSS2); read the press release
17 December 1996	W3C Recommendation of CSS level 1; read the press release
13 August 1996	MSIE 3 browser, with support for CSS
3 July 1996	Birth of Dolly the sheep
27 November 1995	Experimental support for CSS in W3C's Arena browser
6-7 November 1995	W3C Workshop on Style Sheets in Rocquencourt, France
24 August 1995	Release of Windows 95
10 October 1994	Håkon's e-mail to www-talk@w3.org

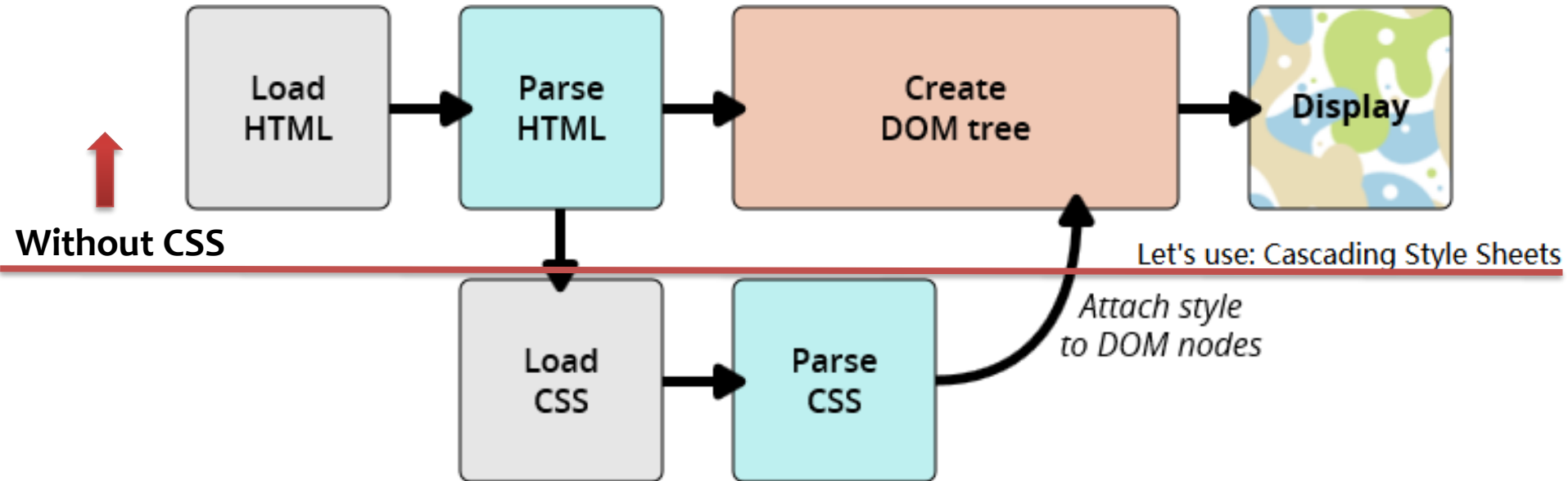
详细信息参见：20 Years of CSS, <https://www.w3.org/Style/CSS20/>

CSS是如何工作的

```
<p>
  Let's use:
  <span>Cascading</span>
  <span>Style</span>
  <span>Sheets</span>
</p>
```

```
P
├ "Let's use:"
├ SPAN
│   └ "Cascading"
├ SPAN
│   └ "Style"
└ SPAN
    └ "Sheets"
```

DOM是一种树形结构. 标记语言中的每个元素,属性,文本片段都变为一个DOM节点。节点的关系有父子、兄弟等

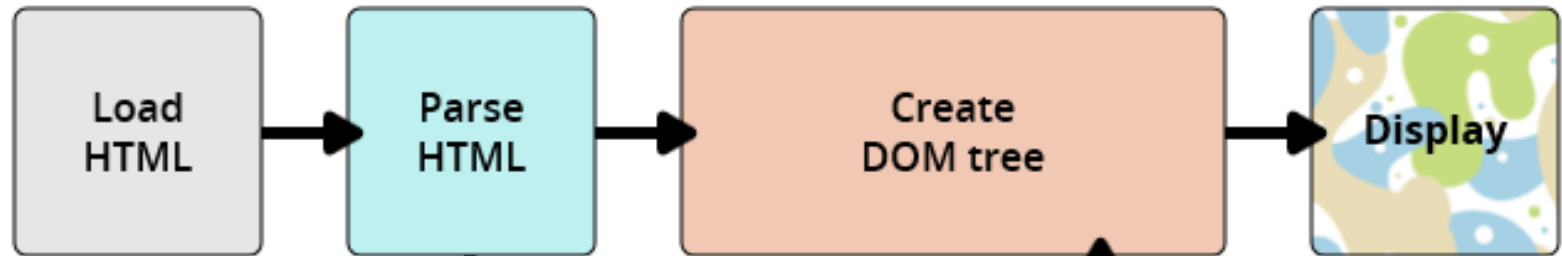


CSS是如何工作的

```
<p>
  Let's use:
  <span>Cascading</span>
  <span>Style</span>
  <span>Sheets</span>
</p>
```

```
P
├ "Let's use:"
├ SPAN
│   └ "Cascading"
├ SPAN
│   └ "Style"
└ SPAN
    └ "Sheets"
```

DOM是一种树形结构. 标记语言中的每个元素,属性,文本片段都变为一个DOM节点。节点的关系有父子、兄弟等



```
<link rel="stylesheet" href="style.css">
```

Let's use: Cascading Style Sheets

Attach style
to DOM nodes

Let's use: Cascading Style Sheets

```
span {
  border: 1px solid black;
  background-color: lime;
}
```

浏览器渲染引擎会通过计算,将对应的CSS声明应用到页面的每一个元素上,从而使得元素们以适当的方式布局,并展示出适当的样式

With CSS

CSS的三种引用方式

- 外部样式表

- 在文件中单独定义，并在HTML的<head>元素中使用<link>来引用
- 当样式需要被应用到多个页面时，使用外部样式表是理想的选择，**quick and clean**

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>My CSS experiment</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Hello World!</h1>
    <p>This is my first CSS example</p>
  </body>
</html>
```

style.css

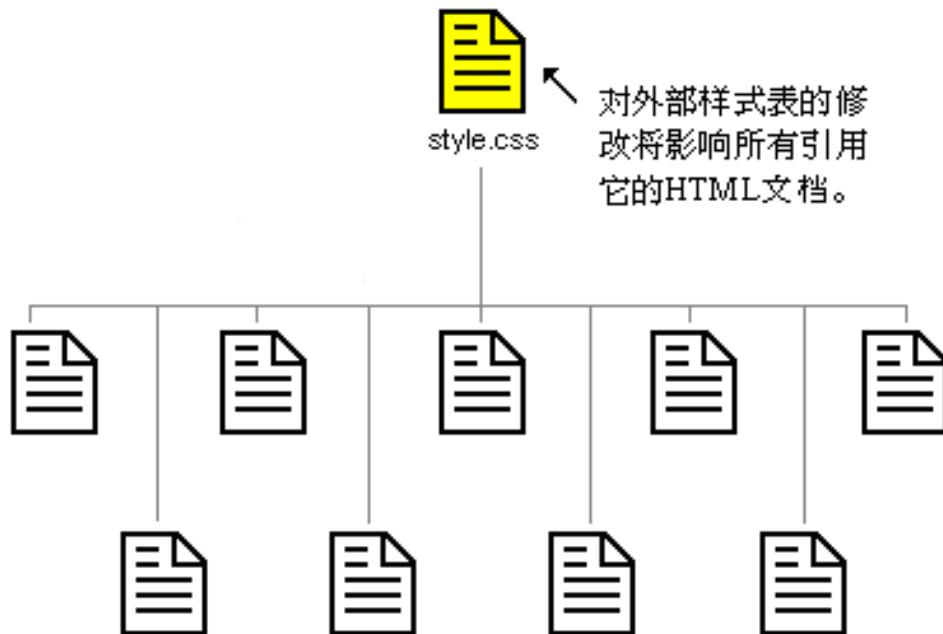
```
h1 {
  color: blue;
  background-color: yellow;
  border: 1px solid black;
}

p {
  color: red;
}
```

CSS的三种引用方式

- 外部样式表

- 在文件中单独定义，并在HTML的<head>元素中使用<link>来引用
- 当样式需要被应用到多个页面时，使用外部样式表是理想的选择，**quick and clean**



对外部样式表的修改将影响所有引用它的HTML文档。

引用同一个样式表的所有HTML文档

CSS的三种引用方式

- 内部样式表

- HTML代码和CSS代码在一个文件中，CSS代码在HTML<head>元素中的<style></style>标签对内
- 不如外部样式表高效，会增加每个网页的大小和加载时间，让用户等待更长时间
- 在某些特殊情况下使用，eg：在使用一个内容管理系统，修改CSS文件可能比较困难的情况

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>CSS Example</title>
<style>
p{
    color:blue;
    text-align:center;
}
</style>
</head>
```

CSS的三种引用方式

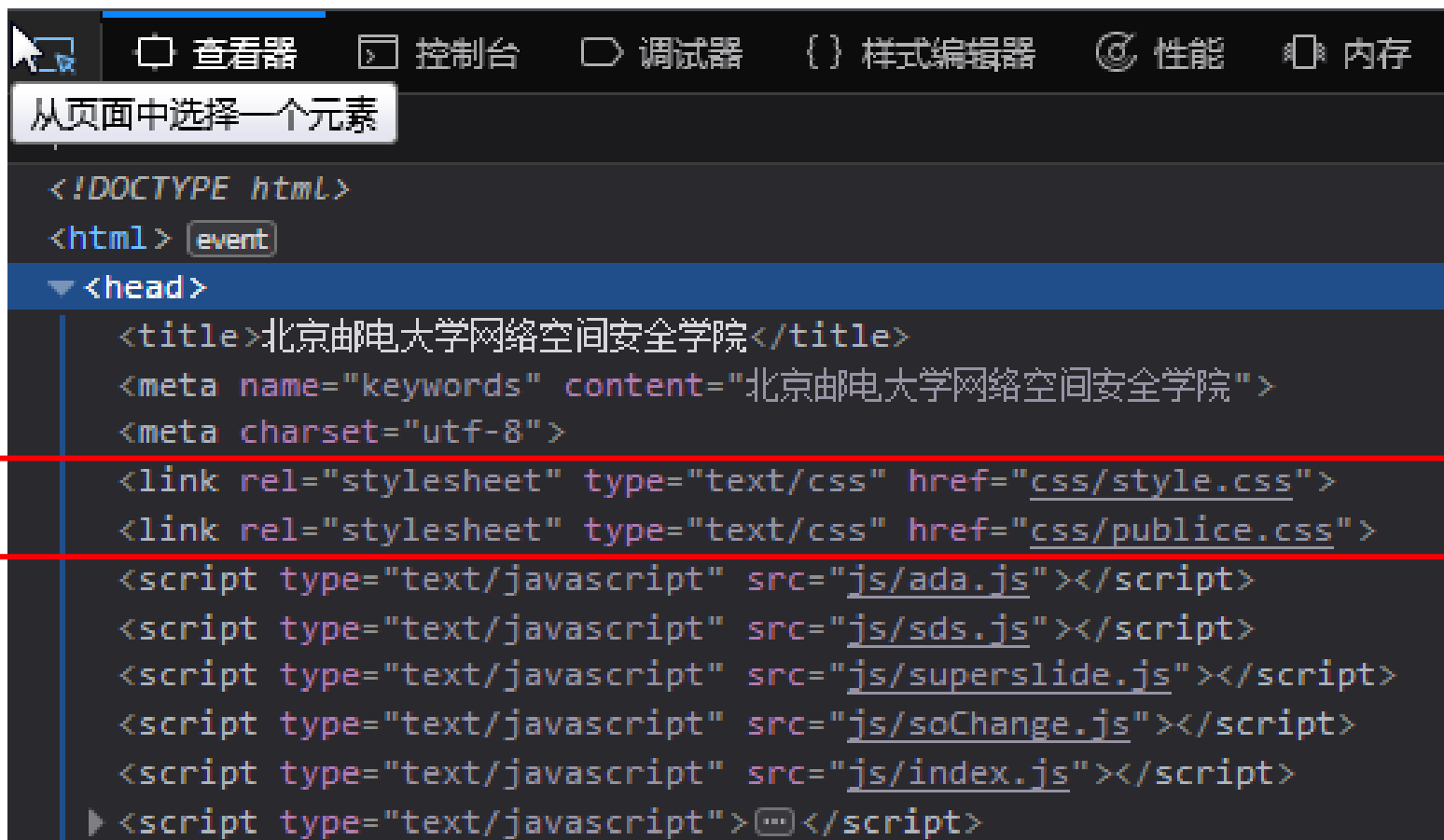
- 内联 (inline) 样式

- HTML代码和CSS代码在一个文件中，CSS代码仅影响一个元素，在元素的style属性中定义
- 冗余代码多，很难维护，混合了CSS表示的样式信息和HTML的结构信息，使CSS难以阅读和理解
- 在某些特殊情况下使用，eg：内容管理系统只允许编辑HTML的body部分

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>My CSS experiment</title>
  </head>
  <body>
    <h1 style="color: blue;background-color: yellow;border: 1px solid black;">Hello World!</h1>
    <p style="color:red;">This is my first CSS example</p>
  </body>
</html>
```

身边的CSS实例

- 北邮网络空间安全学院
 - <https://scss.bupt.edu.cn/index.htm>

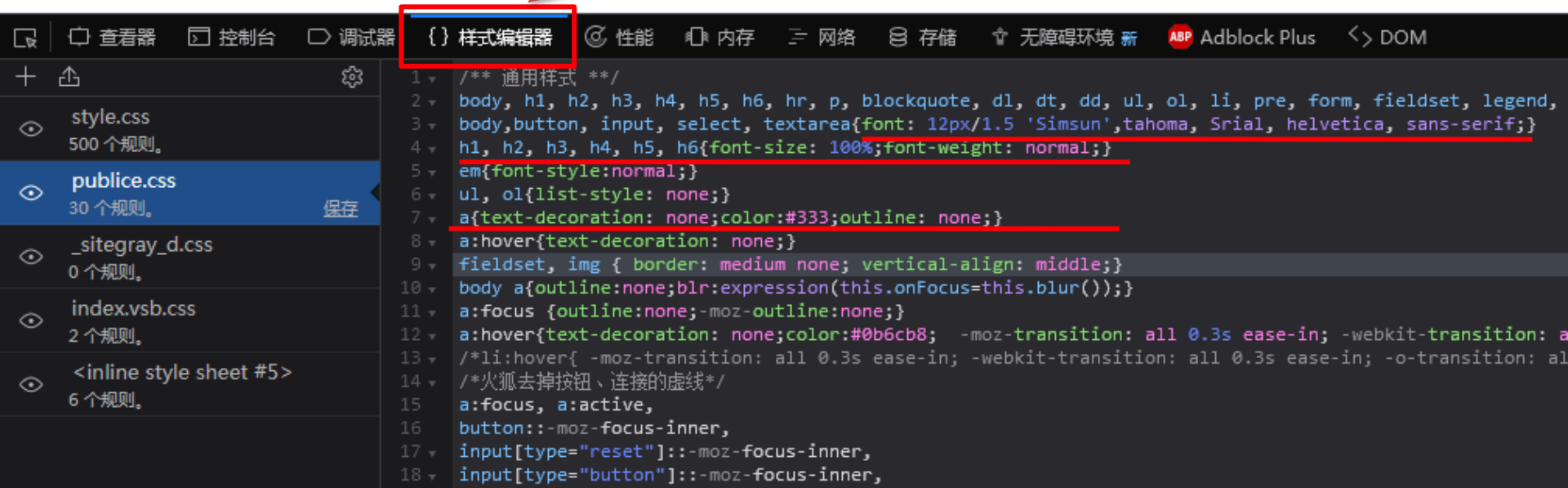


```
<!DOCTYPE html>
<html> event
  <head>
    <title>北京邮电大学网络空间安全学院</title>
    <meta name="keywords" content="北京邮电大学网络空间安全学院">
    <meta charset="utf-8">
    <link rel="stylesheet" type="text/css" href="css/style.css">
    <link rel="stylesheet" type="text/css" href="css/public.css">
    <script type="text/javascript" src="js/ada.js"></script>
    <script type="text/javascript" src="js/sds.js"></script>
    <script type="text/javascript" src="js/superslide.js"></script>
    <script type="text/javascript" src="js/soChange.js"></script>
    <script type="text/javascript" src="js/index.js"></script>
    <script type="text/javascript"> </script>
```

身边的CSS实例

- 北邮网络空间安全学院

– <https://scss.b3log.org/index.htm>



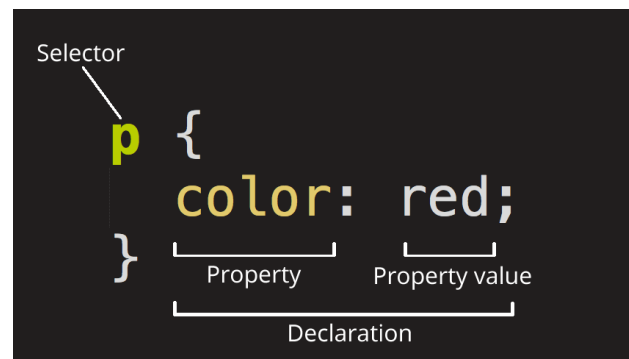
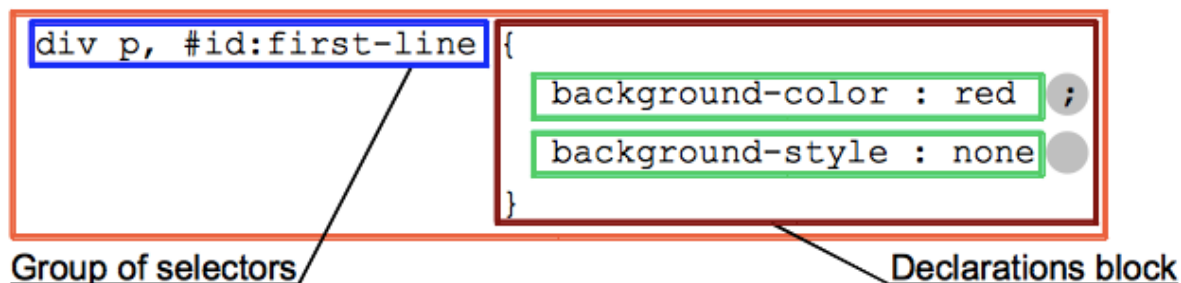
发现： CSS文件是由一条条的CSS规则构成，这些CSS规则定义了HTML元素的样式（如字号、字体，颜色、边界样式、动画效果等）

问题：怎么定义特定元素的样式？我可以控制的样式都有哪些？定义在同一元素上的多个样式如何生效？

CSS语法

- CSS文件是由一条条CSS规则构成的
- CSS 规则由选择器及声明块两个主要的部分构成：
 - 选择器 (Selector) 是一种模式，它能在页面上匹配一些元素，使得相关的声明仅被应用到被选择的元素上
 - 声明块包括一条或多条声明，每条声明由一个属性和一个值组成，属性 (Property) 是希望设置的样式属性 (style attribute)

A CSS ruleset (or rule):



CSS语法

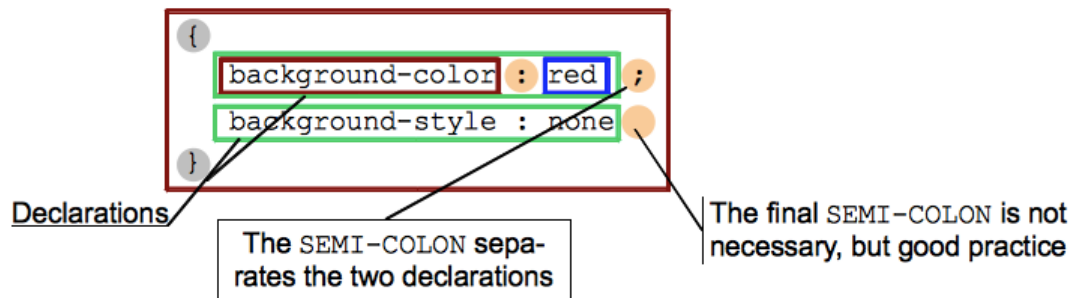
- 到底什么是选择器？

- 选择器提供了一种方式把你想要的那一个元素选中，把它选中了，你才能操作这个元素的CSS样式
- CSS有很多把元素选中的方式，这些不同的方式就是不同的选择器
- 学习CSS就是要正确地选中想要的元素，正确地设定样式

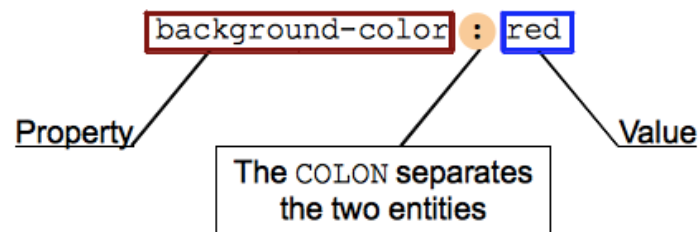
CSS语法

- 每个规则（除了选择器的部分）都应该包含在成对的大括号里（{ }）
- 在每个声明里应该用冒号（:）分离开属性与属性值
- 在每个规则集里，应该用分号（;）分离开各个声明

A CSS declarations block:



A CSS declaration :



提示: 最后一条规则是不需要加分号的，因为分号在英语中是一个分隔符号，不是结束符号。然而，大多数有经验的设计师会在每条声明的末尾都加上分号，这么做的好处是，当你从现有的规则中增减声明时，会尽可能地减少出错的可能性

CSS语法

- 通过添加**空格空行**使样式表更具可读性

```
body {  
  font: 1em/150% Helvetica, Arial, sans-serif;  
  padding: 1em;  
  margin: 0 auto;  
  max-width: 33em;  
}
```

推荐每个声明（以及规则的开始/结束）都在自己各自的行上，方便维护和理解

- CSS中的**注释**以 `/*` 开始并以 `*/` 结束
- 简写属性**（如 `font`，`background`，`padding`，`border`、`margin`等）允许在一行设置多个属性，节省时间并使代码更整洁

```
padding-top: 10px;  
padding-right: 15px;  
padding-bottom: 15px;  
padding-left: 5px;
```

padding: 10px 15px 15px 5px;

顺时针：上→右→下→左

- CSS 对大小写不敏感，存在**例外**是选择器中的**class** 和 **id** 名称对大小写敏感，**推荐全小写**（就像在XHTML标准中要求的那样）

接下来主要内容

- 选择器
- 样式
- 层叠和继承
- 盒模型
- 布局

接下来主要内容

- 选择器
- 样式
- 层叠和继承
- 盒模型
- 布局

CSS选择器

- 选择器是CSS规则的一部分，用于定位我们要样式化的HTML元素，选择器可以被分为以下类别：
 - 简单选择器 (Simple selectors) :
 - 通过元素类型、class 或 id 匹配一个或多个元素
 - 属性选择器 (Attribute selectors) :
 - 通过属性/属性值匹配一个或多个元素

CSS选择器

- 选择器是CSS规则的一部分，用于定位我们要样式化的HTML元素，选择器可以被分为以下类别：
 - 伪类 (Pseudo-classes) :
 - 匹配处于确定状态的一个或多个元素，例如被鼠标指针悬停的元素，或当前被选中或未选中的复选框，或元素是DOM树中一父节点的第一个子节点
 - 伪元素 (Pseudo-elements) :
 - 匹配处于相关的确定位的一个或多个元素，例如每个段落的第一个字，或者某个元素之前生成的内容

CSS选择器

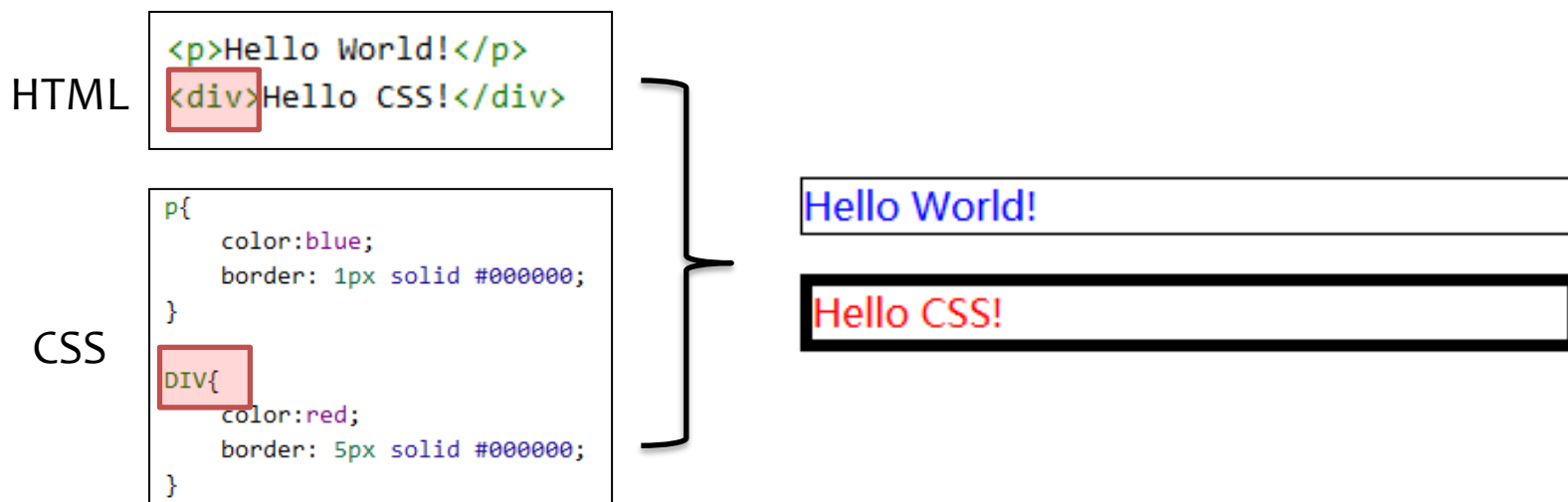
- 选择器是CSS规则的一部分，用于定位我们要样式化的HTML元素，选择器可以被分为以下类别：
 - 组合选择器 (Combinators) :
 - 以有效的方式组合两个或更多的选择器用于实现特定选择的方法，例如，只选择divs的直系子节点的段落，或者直接跟在headings后面的段落
 - 多重选择器 (Multiple selectors) :
 - 将以逗号分开的多个选择器放在一个CSS规则下面，以将一组声明应用于由这些选择器选择的所有元素

简单选择器

- 基于元素的类型（或其 class 或 id）直接匹配文档的一个或多个元素
 - 元素选择器
 - 标识(id)选择器
 - 类(class)选择器

简单选择器

- 基于元素的类型（或其 class 或 id）直接匹配文档的一个或多个元素
 - 元素选择器：选中所有元素名和选择器名匹配的元素（不区分大小写）



简单选择器

- 基于元素的类型（或其 class 或 id）直接匹配文档的一个或多个元素
 - 类(class)选择器、标识(id)选择器 → 元素的id和class属性

元素的id属性

- 用于标识网页元素的唯一身份，在同一HTML页面中，id应该是唯一的，在不同页面是可以出现相同id的元素
- 主要作用是帮助CSS选择器或者JavaScript选中唯一元素
- id也可以写成name，区别在于name是HTML中的标准，而id是XHTML中的标准，现在网页的标准都是使用id，所以大家尽量不要用name属性

```
<div id="first">绿叶学习网</div>  
<p id="first">绿叶学习网</p>
```



简单选择器

- 基于元素的类型（或其 class 或 id）直接匹配文档的一个或多个元素
 - 类(class)选择器、标识(id)选择器 → 元素的 id 和 class 属性

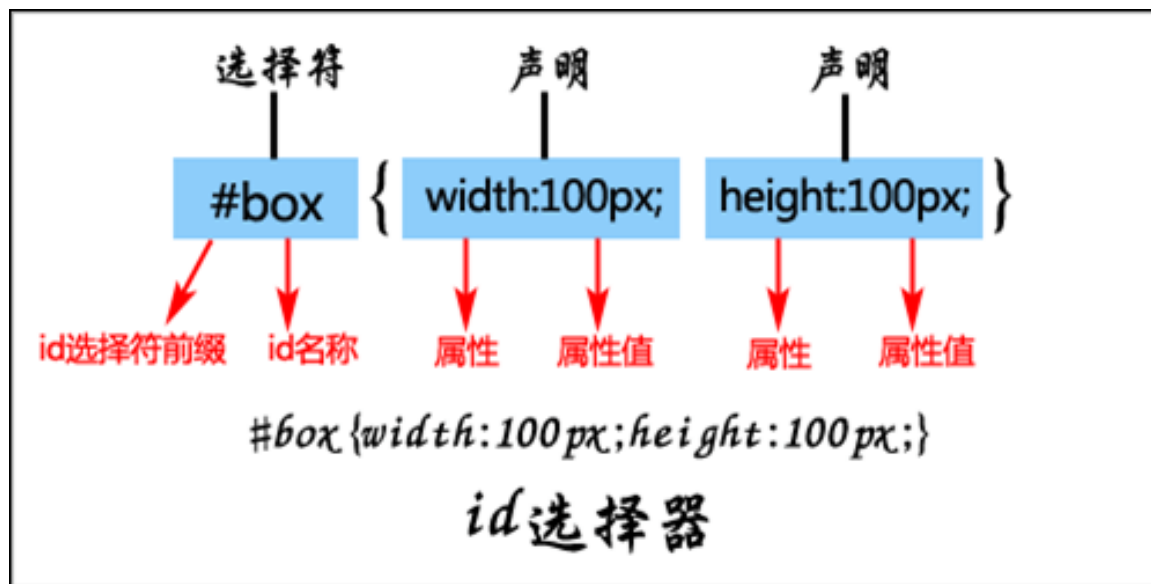
元素的 class 属性

- 定义元素的类名称。同一元素可以设置多个类名，用空格分隔。同一页面，相同或不同的元素可以具有相同的类名。类名不能以数字开头
- 可以为同一个页面的相同元素或者不同元素设置相同的 class，CSS 选择器通过 class 属性选择具有同一属性值的多个元素，使其具有相同的 CSS 样式

```
<ul>
  <li class="first done">Create an HTML document</li>
  <li class="second done">Create a CSS style sheet</li>
  <li class="third">Link them all together</li>
</ul>
```

简单选择器

- 基于元素的类型（或其 class 或 id）直接匹配文档的一个或多个元素
 - 标识(id)选择器：由一个“#”及元素的id属性值组成，用于选中所有id属性值和选择器名匹配的元素（区分大小写）



简单选择器

- 基于元素的类型（或其 class 或 id）直接匹配文档的一个或多个元素
 - 标识(id)选择器：由一个“#”及元素的id属性值组成，用于选中所有id属性值和选择器名匹配的元素（区分大小写）

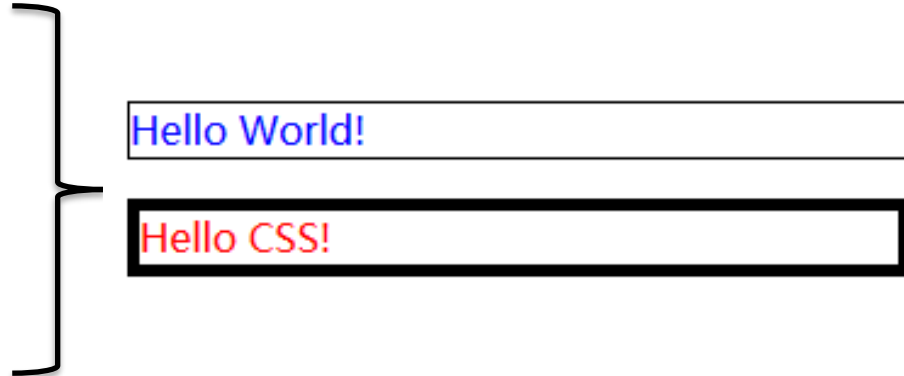
HTML

```
<p id="world">Hello World!</p>  
<div id="CSS">Hello CSS!</div>
```

CSS

```
#world{  
  color:blue;  
  border: 1px solid #000000;  
}  
#CSS{  
  color:red;  
  border: 5px solid #000000;  
}
```

The Fo

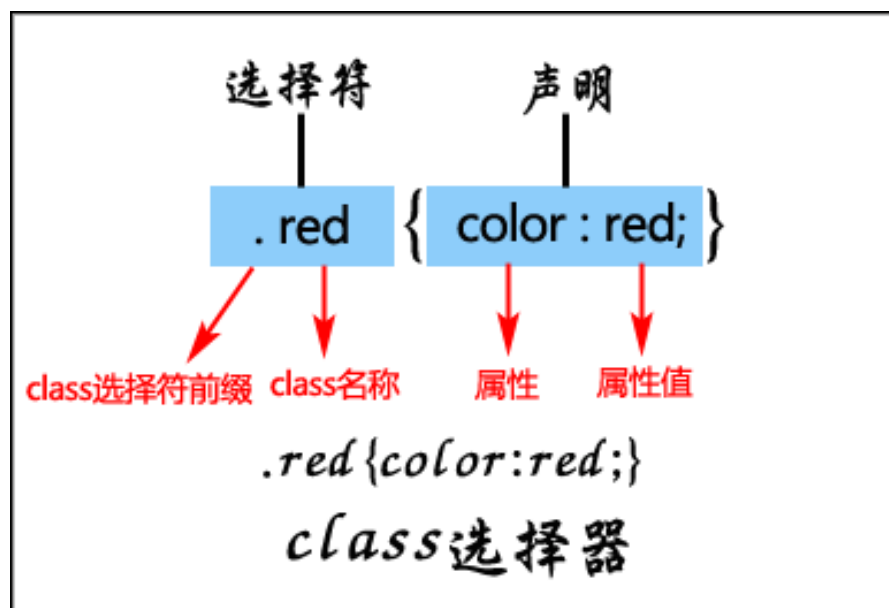


Hello World!

Hello CSS!

简单选择器

- 基于元素的类型（或其 class 或 id）直接匹配文档的一个或多个元素
 - 类(class)选择器：由一个“.”及元素的类名属性值组成。类名是class属性中**没有空格**的任何值，用于选中所有类名和选择器名匹配的元素（**区分大小写**）



简单选择器

- 基于元素的类型（或其 class 或 id）直接匹配文档的一个或多个元素
 - 类(class)选择器：由一个“.”及元素的类名属性值组成。类名是class属性中**没有空格**的任何值，用于选中所有类名和选择器名匹配的元素（**区分大小写**）

HTML

```
<p class="world">Hello World!</p>  
<div class="CSS">Hello CSS!</div>
```

CSS

```
.world{  
    color:blue;  
    border: 1px solid #000000;  
}  
.CSS{  
    color:red;  
    border: 5px solid #000000;  
}
```

The Fol



Hello World!

Hello CSS!

简单选择器

- 基于元素的类型（或其 class 或 id）直接匹配文档的一个或多个元素
 - 类(class)选择器：由一个 “.” 及元素的类名属性值组成。类名是class属性中 **没有空格** 的任何值，用于选中所有类名和选择器名匹配的元素（**区分大小写**）

`element.className`

更加常用的方式

Syntax	Example	Example described
<code>element.className</code>	<code>p.abc</code>	<code><p></code> elements with abc class

HTML

```
<h1 class="hw">Homework 0</h1>
<p class="hw">Due Fri</p>
<p class="hw">Late cutoff Sun</p>
<h1>Lectures</h1>
<p>Apr 3: Syllabus</p>
<p>Apr 5: HTML+CSS</p>
```

CSS

```
p.hw {
  color: green;
}
```

Homework 0

Due Fri

Late cutoff Sun

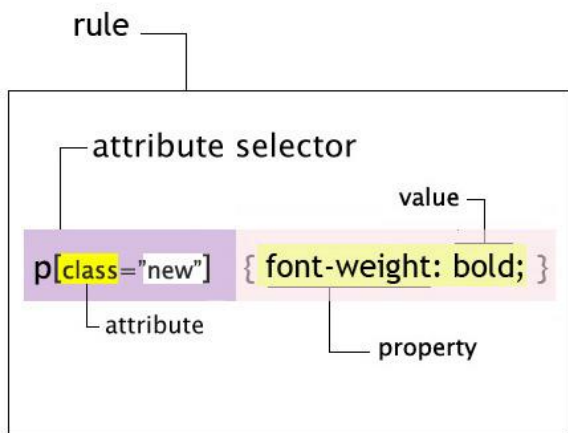
Lectures

Apr 3: Syllabus

Apr 5: HTML+CSS

属性选择器

- 属性选择器是一种特殊类型的选择器，它根据元素的**属性和属性值来匹配元素**
- 通用语法由方括号([])组成，其中包含属性名称，后跟可选条件以匹配属性的值



- ATTRIBUTE
- ATTRIBUTE VALUE
- ATTRIBUTE SELECTOR
- DECLARATION
- DECLARATION BLOCK

属性选择器示例：

```
a[href] {color:red;}
a[href][title] {color:red;}
a[href="http://www.w3school.com.cn/about_us.asp"] {color: red;}
p[class="important warning"] {color: red;}
img[title~="Figure"] {border: 1px solid gray;}
*[lang|="en"] {color: red;}
a[href*="w3school.com.cn"] {color: red;}
```

Copyright © 2010 CSS Bakery www.css-bakery.com * All Rights Reserved

属性选择器

- 属性选择器是一种特殊类型的选择器，它根据元素的属性和属性值来匹配元素
- 根据其匹配的方式可以分为如下两类：
 - 根据属性的存在或属性值进行精确匹配
 - 根据属性值的子串进行类正则表达式的非精确匹配

属性选择器

- 属性选择器是一种特殊类型的选择器，它根据元素的属性和属性值来匹配元素
- 根据其匹配的方式可以分为如下两类：
 - 根据属性的存在或属性值进行精确匹配

```
<p class="Hello world">Hello World!</p>
<p id="html">Hello HTML!</p>
<p class="Hello CSS">Hello CSS!</p>
```

- ① **[attr]** 选择带有以 attr 命名的属性的元素
- ② **[attr=value]** 选择带有以 attr 命名的，且值为 "value" 的属性的元素
- ③ **[attr~=value]** 选择带有以 attr 命名的属性的元素，并且该属性是一个以空格作为分隔的值列表，其中至少一个值为 "value"

```
[class]{
  color:blue;
  border: 1px solid #000000;
}
[id="html"]{
  color:green;
  border: 3px solid #000000;
}
[class~="world"]{ 区分大小写
  color:red;
  border: 5px solid #000000;
}
```

Hello World!

Hello HTML!

Hello CSS!

属性选择器

- 属性选择器是一种特殊类型的选择器，它根据元素的属性和属性值来匹配元素
- 根据其匹配的方式可以分为如下两类：
 - 根据属性值的子串进行类正则表达式的非精确匹配

```
<p class="Hello world">Hello World!</p>
<p id="html">Hello HTML!</p>
<p class="Hello CSS">Hello CSS!</p>
```

① **[attr=val]** 选择attr属性的值是val 或值以 val- 开头的元素

② **[attr^=val]** 选择attr属性的值以val 开头（包括 val）的元素

③ **[attr\$=val]** 选择attr属性的值以val 结尾（包括 val）的元素

④ **[attr*=val]** 选择attr属性的值中包含子字符串 val 的元素

```
[id^="ht"]{
  color:green;
  border: 3px solid #000000;
}
[class$="world"]{
  color:blue;
  border: 1px solid #000000;
}
[class*="CSS"]{
  color:red;
  border: 5px solid #000000;
}
```

Hello World!

Hello HTML!

Hello CSS!

属性选择器

- 属性选择器是一种特殊类型的选择器，它根据元素的属性和属性值来匹配元素
- 根据其匹配的方式可以分为如下两类：
 - 根据属性值的子串进行类正则表达式的非精确匹配

① **[attr=val]** 选择attr属性的值是val 或值以val- 开头的元素

② **[attr^=val]** 选择attr属性的值以val 开头（包括val）的元素

③ **[attr\$=val]** 选择attr属性的值以val 结尾（包括val）的元素

④ **[attr*=val]** 选择attr属性的值中包含子字符串val 的元素

```
<head>
<style type="text/css">
  [title][title|= "He"]
  {
    color:red;
  }
</style>
</head>
```

可以应用样式:

Hello world

[W3School](http://w3school.com.cn)

```
<body>
<h1>可以应用样式: </h1>
<h2 title="He-llo world">Hello world</h2>
<a title="W3School" href="http://w3school.com.cn">W3School</a>
```

问题：如果把He-llo改成Hello，Hello world的颜色会是什么样子的呢？

答案：不会被选中

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾，用来选择**特定状态下的元素**
- 伪元素 (pseudo-element) 选择器以双冒号(::)为分割添加到选择器末尾，用来选择**元素的某个特定部分**

```
selector:pseudo-class {  
  property: value;  
}
```

伪类 (pseudo-class) 选择器

```
selector::pseudo-element {  
  property: value;  
}
```

伪元素 (pseudo-element) 选择器

提示: 通过引入伪类和伪元素，可以实现不仅根据**文档DOM树**中的内容对元素应用样式，还可以根据诸如像导航历史这样的**外部因素**来应用样式（例如 [:visited](#)），可以根据元素的**状态**（例如在一些表单元素上的 [:checked](#)），或者鼠标的**位置**（例如 [:hover](#) 让你知道是否鼠标在一个元素上悬停）来应用样式

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 状态伪类

- :link伪类表示链接的正常状态, 选择那些尚未被点过的链接
- :visited伪类选择点过的链接
- :focus用于选择已经通过指针设备、触摸或键盘获得焦点的元素, 在表单里使用得非常多
- :hover伪类在用户指针悬停时生效, 不只适用于链接
- :active伪类选择被鼠标指针或触摸操作“激活的”元素, 只发生在鼠标被按下到被释放的这段时间里

```
p:hover{  
  color:red;  
  border: 5px solid #000000;  
  cursor:pointer;  
}
```



提示: 由于css的优先级问题, 伪类需要按顺序书写, 否则会出现问题

L (link) V (visited) F (focus) H (hover) A (active), **LoVe Fear HAte**

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

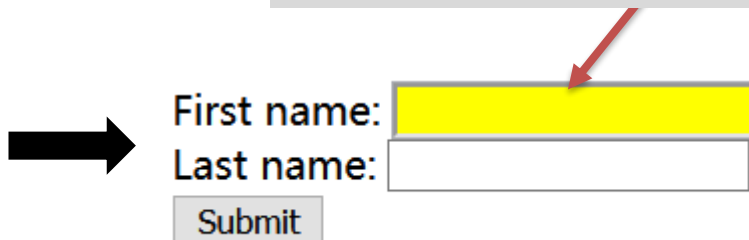
— 状态伪类

- :link伪类表示链接的正常状态, 选择那些尚未被点过的链接
- :visited伪类选择点过的链接
- :hover伪类在用户指针悬停时生效, 不只适用于链接
- :active伪类选择被鼠标指针或触摸操作“激活的”元素, 只发生在鼠标被按下到被释放的这段时间里
- :focus用于选择已经通过指针设备、触摸或键盘获得焦点的元素, 在表单里使用得非常多

```
<!DOCTYPE html>
<html>
<head>
<style type="text/css">
input:focus {background-color:yellow;}
</style>
</head>

<body>
  <form action="form_action.asp" method="get">
    First name: <input type="text" name="fname" /><br />
    Last name: <input type="text" name="lname" /><br />
    <input type="submit" value="Submit" />
  </form>
</body>
</html>
```

输入框获得焦点, 背景色变成黄色



伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-child伪类选择是父元素第一个子元素的指定类型元素
- :last-child伪类选择是父元素最后一个子元素的指定类型元素
- :nth-last-child选择是父元素倒数第n个子元素的指定类型元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
p:first-child {font-weight: bold;}
li:first-child {text-transform:uppercase;}
p:last-child {color: red;}
</style>
</head>

<body>
<div>
<p>These are the necessary steps:</p>
<ul>
<li>Intert Key</li>
<li>Turn key <strong>clockwise</strong></li>
<li>Push accelerator</li>
</ul>
<p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
</div>
<p><b>注释: </b>必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效.</p>
</body>
</html>
```

①p元素, 是其父元素的第一个子元素

②li元素, 是其父元素的第一个子元素

③p元素, 是其父元素的最后一个子元素

These are the necessary steps:

- INTERT KEY
- Turn key **clockwise**
- Push accelerator

Do *not* push the brake at the same time as the accelerator.

注释: 必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-child伪类选择是父元素第一个子元素的指定类型元素
- :last-child伪类选择是父元素最后一个子元素的指定类型元素
- :nth-last-child选择是父元素倒数第n个子元素的指定类型元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
li:nth-last-child(2) {color:green;}
</style>
</head>
<body>
  <div>
    <p>These are the necessary steps:</p>
    <ul>
      <li>Intert Key</li>
      <li>Turn key <strong>clockwise</strong></li>
      <li>Push accelerator</li>
    </ul>
    <p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
  </div>
</body>
</html>
```

These are the necessary steps:

- Intert Key
- Turn key **clockwise**
- Push accelerator

Do *not* push the brake at the same time as the accelerator.

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-of-type伪类选择是父元素指定类型子元素中第一个元素的元素
- :last-of-type伪类选择是父元素指定类型子元素中最后一个元素的元素
- :nth-of-type伪类选择父元素指定类型子元素中的第n个子元素
- :nth-last-of-type伪类选择父元素指定类型子元素中倒数第n个子元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
p:first-of-type {font-weight: bold;}
ul:first-of-type {color: green;}
li:nth-of-type(2) {text-transform:uppercase;}
p:last-of-type {color: red;}
p:nth-last-of-type(2) {background-color: yellow;}
</style>
</head>
<body>
<div>
<p>These are the necessary steps:</p>
<ul>
<li>Intert Key</li>
<li>Turn key <strong>clockwise</strong></li>
<li>Push accelerator</li>
</ul>
<p>Here is an example</p>
<p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
</div>
<p><b>注释: </b>必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效.</p>
</body>
</html>
```

These are the necessary steps:

- Intert Key
- TURN KEY **CLOCKWISE**
- Push accelerator

Here is an example

Do *not* push the brake at the same time as the accelerator.

注释: 必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-of-type伪类选择是父元素指定类型子元素中第一个元素的元素
- :last-of-type伪类选择是父元素指定类型子元素中最后一个元素的元素
- :nth-of-type伪类选择父元素指定类型子元素中的第n个子元素
- :nth-last-of-type伪类选择父元素指定类型子元素中倒数第n个子元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
p:first-of-type {font-weight: bold;}
ul:first-of-type {color: green;}
li:nth-of-type(2) {text-transform: uppercase;}
p:last-of-type {color: red;}
p:nth-last-of-type(2) {background-color: yellow;}
</style>
</head>
<body>
<div>
<p>These are the necessary steps:</p>
<ul>
<li>Intert Key</li>
<li>Turn key <strong>clockwise</strong></li>
<li>Push accelerator</li>
</ul>
<p>Here is an example</p>
<p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
</div>
<p><b>注释: </b>必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。</p>
</body>
</html>
```

These are the necessary steps:

- Intert Key
- TURN KEY **CLOCKWISE**
- Push accelerator

Here is an example

Do *not* push the brake at the same time as the accelerator.

注释: 必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-of-type伪类选择是父元素指定类型子元素中第一个元素的元素
- :last-of-type伪类选择是父元素指定类型子元素中最后一个元素的元素
- :nth-of-type伪类选择父元素指定类型子元素中的第n个子元素
- :nth-last-of-type伪类选择父元素指定类型子元素中倒数第n个子元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
p:first-of-type {font-weight: bold;}
ul:first-of-type {color: green;}
li:nth-of-type(2) {text-transform:uppercase;}
p:last-of-type {color: red;}
p:nth-last-of-type(2) {background-color: yellow;}
</style>
</head>
<body>
<div>
<p>These are the necessary steps:</p>
<ul>
<li>Intert Key</li>
<li>Turn key <strong>clockwise</strong></li>
<li>Push accelerator</li>
</ul>
<p>Here is an example</p>
<p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
</div>
<p><b>注释: </b>必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。</p>
</body>
</html>
```

These are the necessary steps:

- Intert Key
- TURN KEY **CLOCKWISE**
- Push accelerator

Here is an example

Do *not* push the brake at the same time as the accelerator.

注释: 必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-of-type伪类选择是父元素指定类型子元素中第一个元素的元素
- :last-of-type伪类选择是父元素指定类型子元素中最后一个元素的元素
- :nth-of-type伪类选择父元素指定类型子元素中的第n个子元素
- :nth-last-of-type伪类选择父元素指定类型子元素中倒数第n个子元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
p:first-of-type {font-weight: bold;}
ul:first-of-type {color: green;}
li:nth-of-type(2) {text-transform:uppercase;}
p:last-of-type {color: red;}
p:nth-last-of-type(2) {background-color: yellow;}
</style>
</head>
<body>
<div>
<p>These are the necessary steps:</p>
<ul>
<li>Intert Key</li>
<li>Turn key <strong>clockwise</strong></li>
<li>Push accelerator</li>
</ul>
<p>Here is an example</p>
<p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
</div>
<p><b>注释: </b>必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。</p>
</body>
</html>
```

These are the necessary steps:

- Intert Key
- TURN KEY **CLOCKWISE**
- Push accelerator

Here is an example

Do *not* push the brake at the same time as the accelerator.

注释: 必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :first-of-type伪类选择是父元素指定类型子元素中第一个元素的元素
- :last-of-type伪类选择是父元素指定类型子元素中最后一个元素的元素
- :nth-of-type伪类选择父元素指定类型子元素中的第n个子元素
- :nth-last-of-type伪类选择父元素指定类型子元素中倒数第n个子元素

```
<!DOCTYPE HTML>
<html>
<head>
<style type="text/css">
p:first-of-type {font-weight: bold;}
ul:first-of-type {color: green;}
li:nth-of-type(2) {text-transform:uppercase;}
p:last-of-type {color: red;}
p:nth-last-of-type(2) {background-color: yellow;}
</style>
</head>
<body>
<div>
<p>These are the necessary steps:</p>
<ul>
<li>Intert Key</li>
<li>Turn key <strong>clockwise</strong></li>
<li>Push accelerator</li>
</ul>
<p>Here is an example</p>
<p>Do <em>not</em> push the brake at the same time as the accelerator.</p>
</div>
<p><b>注释: </b>必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。</p>
</body>
</html>
```

These are the necessary steps:

- Intert Key
- TURN KEY **CLOCKWISE**
- Push accelerator

Here is an example

Do *not* push the brake at the same time as the accelerator.

注释: 必须声明 DOCTYPE, 这样 :first-child 才能在 IE 中生效。

伪类和伪元素选择器

- 伪类（pseudo-class）选择器以冒号(:)为分割添加到选择器末尾，用来选择特定状态下的元素

— 区别

```
<div>
  <h2>这是标题</h2>
  <p>这是段落一</p>
  <p>这是段落二</p>
</div>
```

```
p:first-child
{
  color: red;
}
```

这是标题

这是段落一

这是段落二

```
p:first-of-type {
  color: red;
}
```

这是标题

这是段落一

这是段落二

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :nth-child伪类根据元素在标记中的次序选择相应的元素
- :only-child选择是父元素中唯一子元素的所有元素
- :not伪类选择与参数不匹配的所有元素
- :only-of-type选择是父元素的指定类型唯一子元素的所有元素

```
<ul>
  <li class="link"><a href="#">United Kingdom</a></li>
  <li class="link"><a href="#">Germany</a></li>
  <li class="link"><a href="#">Finland</a></li>
  <li class="link"><a href="#">Russia</a></li>
  <li class="link2"><a href="#">Spain</a></li>
  <li class="link"><a href="#">Poland</a></li>
</ul>
```



```
li:nth-of-type(2n) {
  background-color: grey;
}
li:nth-of-type(2n+1) {
  background-color: orange;
}
li:not([class="link"]){
  background-color: brown;
}
```

- [United Kingdom](#)
- [Germany](#)
- [Finland](#)
- [Russia](#)
- [Spain](#)
- [Poland](#)

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- `:nth-child`伪类根据元素在标记中的次序选择相应的元素
- `:only-child`选择是父元素中唯一子元素的所有元素
- `:not`伪类选择与参数不匹配的所有元素
- `:only-of-type`选择在同一个父元素中没有其他相同类型兄弟元素的元素

```
<!DOCTYPE html>
<html>
<head>
<style>
p:only-of-type{background:red;}
em:only-child{background:yellow;}
</style>
</head>
<body>
  <div> <p>这是一个段落。</p> </div>
  <div>
    <p>这是一个段落。</p>
    <p>这是一个段落。</p>
  </div>
  <div> <em>这是一个段落。</em> </div>
</body>
</html>
```

这是一个段落。

这是一个段落。

这是一个段落。

这是一个段落。

伪类和伪元素选择器

- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 结构伪类

- :nth-child伪类根据元素在标记中的次序选择相应的元素
- :only-child选择是父元素中唯一子元素的所有元素
- :not伪类选择与参数不匹配的所有元素
- :only-of-type选择是父元素的指定类型唯一子元素的所有元素

```
<div>
  <p>这是段落</p>
  <h2>这是标题</h2>
</div>
```

```
p:only-of-type {
  color: blue;
}
```

这是段落

这是标题

```
p:only-child {
  color: blue;
}
```

这是段落

这是标题

伪类和伪元素选择器

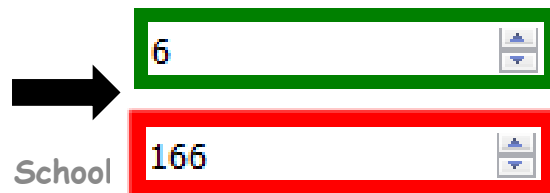
- 伪类 (pseudo-class) 选择器以冒号(:)为分割添加到选择器末尾, 用来选择特定状态下的元素

— 验证伪类

- :checked伪类选择被勾选或选中的单选按钮、多选按钮及列表选项
- :default伪类从表单中选择默认的元素
- :disabled伪类选择禁用状态的表单元素, :enabled
- :empty伪类选择其中不包含任何内容的空元素
- :in-range伪类选择有范围且值在指定范围内的元素, :out-of-range
- :indeterminate伪类选择页面加载时没有勾选的单选按钮或复选框
- :valid伪类选择输入格式符合要求的表单元素, :invalid
- :optional伪类选择表单中非必填的输入字段
- :read-only伪类选择用户不能编辑的元素, :read-write、:required

```
<input type="number" min="5" max="10">
```

```
input[type=number]:in-range { border: 5px solid green; }  
input[type=number]:out-of-range { border: 5px solid red; }
```



伪类和伪元素选择器

- 伪类（pseudo-class）选择器以冒号(:)为分割添加到选择器末尾，用来选择特定状态下的元素

— 语言伪类

- `:lang()` 基于元素语言来匹配页面元素

```
<ol>
  <li lang="en">Manchester United</li>
  <li lang="es">Barcelona</li>
  <li lang="de">Bayern Munich</li>
  <li lang="es">Real Madrid</li>
  <li lang="de">Borussia Dortmund</li>
  <li lang="en">Southampton FC</li>
</ol>
```



```
li:lang("en") {background: pink; font-weight: bold;}
li:lang("de") {background: lightgrey; font-weight: bold;}
li:lang("es") {background: yellow; font-weight: bold;}
```

- 
1. Manchester United
 2. Barcelona
 3. Bayern Munich
 4. Real Madrid
 5. Borussia Dortmund
 6. Southampton FC

课程作业

- 实操性作业（小组作业，2-3人一个小组）
 - 设计并实现一个完整的静态网站
 - 在<head>部分注意标题、元信息等的引入
 - 在<body>部分注意使用段落与文字、列表、表格、多媒体与嵌入、超链接等课题所学内容，注意练习表单
 - 注意考虑网站信息架构设计
 - 提交完整的project，包括网页文件、图片等媒体资源等，注意目录结构、注意语义正确
 - 评价方法：从网页内容的丰富程度、美观、语义等方面打分

课程作业

- 扩展阅读类作业（**每位学生独立完成**），开放选题，和本章节内容相关即可，部分推荐题目如下：
 - HTML的发展历史和HTML5的新特性
 - Web Accessibility, Web无障碍
 - 可参考：<https://zhuanlan.zhihu.com/p/76438798>
 - Web标准化
 - 评价调研深度广度、报告格式（图文并茂，广征博引）
- 作业提交方式：
 - 提交扩展阅读报告（**每人一份**），包括姓名、学号，
 - 提交静态网站源代码（**每组一份,每人均要提交**），包括组员姓名、学号
 - 云课堂，11月2日前提交

推荐课后阅读

- **A friendly web development tutorial for complete beginners**
 - <https://internetingishard.com/html-and-css/>

台湾省对大陆开展的渗透活动

主要职能：5名本部主官，负责资通电军整体工作规划、政治建设等

指挥官：简华庆中将

副指挥官：王岳洋少将

参谋长：陈俊良少将

政战主任：胡锦涛上校

副参谋长：周文祥上校

综合研判部门。主要职能：原陆军六、八、十军团与花东、金门指挥部下属资电部队合并而成，现除资通科外，其余各队工作地点不在本部忠信园区。重点工作为公开渠道重要信息的搜集及综合研判。

资通科

资讯通信支援第一大队

资讯通信支援第二大队

资讯通信支援第三大队

花莲资讯通信作业队

金门资讯通信作业队

通信保障部门。主要职能：全员在本部办公，统合台湾军队网络、电子及指管平台等三大领域，执行网路空间安全维护、电磁频谱侦搜及指管系统建立与运维，重点工作为台军内部通信的保障与维护。

网络技术支持中心

网络作战大队

网络应变中心

网络环境析析中心

指管防护科

技术攻防部门。主要职能：全员在本部办公，工作内容包括电子防护、电子支援、电子攻击，重点工作为网络攻防和电子干扰、反干扰。

第一中队

第二中队

第三中队

花莲电战队

资通电军

网络作战处

电子作战处

训测中心

后勤处

资通电业科



悬赏通告



林嘉宏 男
中国台湾身份证号码
F124293589



周宜庆 男
中国台湾身份证号码
F124832898



顾哲豪 男
中国台湾身份证号码
A120245084



张俊敏 男
中国台湾身份证号码
A122789827



王正平 男
中国台湾身份证号码
B00016865



王圣谦 男
中国台湾身份证号码
F120802257



张洛郡 男
中国台湾身份证号码
A120415658



张子扬 男
中国台湾身份证号码
A120505086



萧佳和 男
中国台湾身份证号码
A124207545



洪瑞瑞 男
中国台湾身份证号码
A120206380



许翔男 男
中国台湾身份证号码
F128575153



吕光耀 男
中国台湾身份证号码
F120216314



何冠威 男
中国台湾身份证号码
B122806124



林玮伦 男
中国台湾身份证号码
D121739129



黄淑卿 女
中国台湾身份证号码
F223450974



吴韵蓉 女
中国台湾身份证号码
K223424505



黄雅瑜 女
中国台湾身份证号码
F227657930

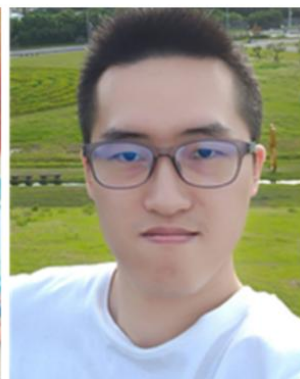
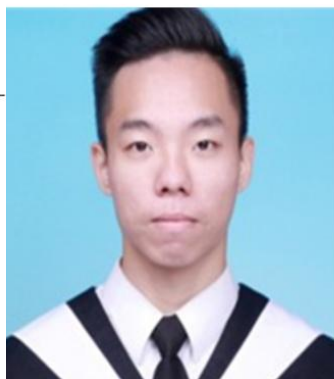


陈奕铭 男
中国台湾身份证号码
A128771470

我局接台湾民众举报称，台湾民进党当局利用架设反动宣传媒介、制作传播反动游戏等手段，对两岸民众进行鼓动、宣传，造成恶劣影响。经查，台湾军方“政治作战局心理作战大队”（下称“心战大队”）长期策划实施煽动分裂国家活动。为依法惩治危害国家主权和领土完整的犯罪行为，切实维护国家安全，我局决定悬赏征集林嘉宏等18名“心战大队”核心骨干违法犯罪嫌疑人。

发现相关线索可立即向我局举报，我局将对举报人身份信息严格保密。凡提供有效线索的举报人，以及配合公安机关抓获犯罪嫌疑人的有功人员，我局将给予1万元人民币的奖励。对打击报复举报人的，将依法严惩。

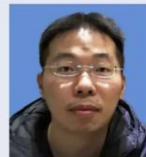
举报电话：0592-110



台湾资通电军现役人员：罗俊铭（左）、洪莉棋（中）、廖韦纶（右）

央视新闻

国家安全部公开 四名“台独”网军信息



林钰书 男
中国台湾身份证号码
P122635546



蔡杰宏 男
中国台湾身份证号码
C121530051



粘孝帆 男
中国台湾身份证号码
B121925951



王浩铭 男
中国台湾身份证号码
T123822994

举报电话：12339

网络举报平台：www.12339.gov.cn

台湾省对大陆开展的渗透活动

台湾资通电军专门组建网络战联队，并雇佣社会黑客、网安公司作为外协力量，执行民进党当局下达的网络作战指令，开展窃密、破坏、反宣活动。他们惯用的手段包括：

渗透大陆水、电、气、热、通信、联网摄像头等关键信息基础设施；

向大陆党政军企重点单位发送钓鱼和反宣电子邮件；

盗取网络直播平台、联网电子显示屏、IP网络对讲广播系统或门户网站的登录账号密码，获取控制权后插播反宣音视频或张贴反宣图片；

潜伏在主流社交媒体平台，豢养大量“机器人”账号，伺机散布虚假消息、操纵舆情走向，误导大陆和台湾民众认知，监控打压岛内异见人士。

自2023年起，又以“匿名者64”组织等名义，在网络社交媒体平台散布虚假消息，竭力为民进党当局谋“独”行径张目。

在网络攻击活动中，资通电军大量使用蚁剑、冰蝎、Metasploit、Quasar等开源工具，其安全防护措施犹如“皇帝的新衣”，在国家安全机关强力调查下一触即溃、暴露无遗。近期，国家安全机关已顺藤摸瓜，查获资通电军在用的数十个网攻平台，发现其针对大陆重要领域实施攻击窃密的新动向，并全部核查处置，斩断窃密“黑手”。